MINISTRY OF HIGHER EDUCATION AND SCIENTIFIC RESEARCH
UNIVERSITY OF LA MANOUBA
NATIONAL SCHOOL OF COMPUTER SCIENCES



GRADUATION PROJECT REPORT

Presented in the purpose of obtaining

Engineering Degree in Computer Science

By

Fares GAALOUL

Subject

Design & Development of a Graphical-Based Machine Learning Pipelines Designer & AutoML Web Application

Hosting Company



Supervised by: Mr. Ilyes KAROUI & Mr. Anis TISSAOUI

Pedagogic adviser: Dr. Rym BESROUR

Address: Lake Gardens, Dollar Street Lake City Center Building Bloc A, Tunis 1053

Phone : +216 71 195 360

Academic year

2019/2020

Supervisors' Signatures

Nom du stagiaire Fares Gaalouf.



Dedication

"To my parents, who have raised me to be the person I am today, You have been with me through every step of the way. Thank you for all the love, guidance, sacrifice and support you have given me. Thank you for everything."

Acknowledgements

The accomplishment of this internship is the result of the contribution and the effective guidance of countless individuals who helped me during this project.

First of all, I want to thank **Mr. Ilyes Karoui** for his continuous support, daily supervision and wise words during this internship. The success of this internship is all thanks to his will to push me to make the best out of this project.

I also want to thank **Mr. Samy Achour**, CEO and founder of Integration Objects, as well as **Mr. Anis Tissaoui**, **Mrs. Mariem Fki** and everyone in the HazeML team for their vision, dedication and contribution to the project.

Sincere gratitude goes to **Dr. Rym Besrour**, my academic tutor, for her availability and words of encouragement.

I would also like to thank for the members of the jury for taking the time to evaluate my work and give me valuable feedback.

I am also grateful for all the friends that I made during this three-year journey at the National School of Computer Science who supported me along this path and with whom I shared countless enlightening experiences.

Finally, many thanks to all the people that had a direct or indirect contribution to the project and were not explicitly mentioned. Your help and support is very much appreciated.

Contents

General Introduction **2**

1 General Context **3**

1.1 Academic Context . . . 3

1.2 Professional Context . . . 3

1.2.1 Presentation of the Hosting Company . . . 4

1.2.1.1 Overview . . . 4

1.2.1.2 Services . . . 4

1.2.2 Staff Organization . . . 4

1.2.3 Work Methodology . . . 5

1.3 Project Context . . . 7

1.3.1 Project Challenges . . . 7

1.3.2 Project Objectives . . . 8

2 Preliminary Study **9**

2.1 State of the Art . . . 9

2.1.1 Machine Learning Pipelines . . . 9

2.1.1.1 Advantages of Machine Learning Pipelines . . . 10

2.1.1.2 Machine Learning Pipeline Steps . . . 10

2.1.2 Automated Machine Learning (AutoML) . . . 12

2.1.2.1 Techniques of AutoML . . . 12

2.1.2.2 Goals of AutoML . . . 16
2.1.2.3 Challenges of AutoML . . . 17
2.1.3 Metalearning . . . 17
2.1.3.1 Metafeatures . . . 18
2.1.3.2 Meta-learners . . . 19
2.2 Study of the existing . . . 20
2.2.1 Existing AutoML frameworks . . . 20
2.2.1.1 Auto-sklearn . . . 20
2.2.1.2 Tree-based Pipeline Optimizer (TPOT) . . . 21
2.2.1.3 H2O AutoML . . . 21
2.2.1.4 Microsoft ML.NET AutoML . . . 21
2.2.2 Benchmark of AutoML frameworks . . . 22
2.2.2.1 Classification benchmarks . . . 22
2.2.2.2 Regression benchmarks . . . 23
2.3 Proposed Solution . . . 24

3 Specification & Requirements **25**

3.1 Requirement Analysis . . . 25
3.1.1 Identifying Actors . . . 25
3.1.2 Functional requirements . . . 26
3.1.3 Non Functional Requirements . . . 26
3.2 Requirements Specification . . . 27
3.2.1 General use case diagram . . . 27
3.2.2 Sequence diagram . . . 28

4 Design **31**

4.1 Global Design . . . 31
4.1.1 General Architecture . . . 31

4.1.1.1 End-to-end pipeline recommendation . . . 32
4.1.1.2 Hyperparameter optimization . . . 36
4.2 Detailed design . . . 37
4.2.1 Sequence diagrams . . . 37
4.2.1.1 End to end pipeline recommendation sequence diagram . . . 37
4.2.1.2 Hyperparameter tuning sequence diagram . . . 39
4.2.2 Activity diagram . . . 40

5 Achievements **42**
5.1 Work Environment . . . 42
5.1.1 Hardware Environment . . . 42
5.1.2 Software environment . . . 43
5.2 Technological choices . . . 43
5.3 Results . . . 44
5.3.1 Metalearning database . . . 44
5.3.2 Illustrations from the Application . . . 48
5.4 Benchmarks . . . 50
5.4.1 Classification benchmarks . . . 51
5.4.1.1 Metrics . . . 51
5.4.1.2 Datasets . . . 51
5.4.1.3 Results . . . 52
5.4.2 Regression benchmarks . . . 55
5.4.2.1 Metrics . . . 55
5.4.2.2 Datasets . . . 55
5.4.2.3 Results . . . 56
5.5 Project planning . . . 57

General Conclusion **60**

Bibliography **62**

Netography **64**

A Software & Technologies **65**

A.1 Software environment 65

A.2 Technologies 66

List of Figures

1.1 Staff organization. 5

1.2 Work methodology overview. 6

2.1 Example of a task-specific machine learning pipeline. 10

2.2 Template of a machine learning pipeline. Differences exist regarding the data cleaning steps. 11

2.3 General architecture of AutoML systems. 12

2.4 Iterations of Bayesian optimization. 13

2.5 Flowchart of genetic algorithms. 14

2.6 Stacked ensembles illustrated. 16

2.7 General process of metalearning, illustrated. 18

2.8 Transforming dataset into fixed-size metafeatures vector. 18

3.1 General use case diagram. 27

3.2 Sequence diagram for pipeline recommendation. 29

4.1 Global architecture. 32

4.2 Automated generation of the search space from a given pipeline. 36

4.3 End to end pipeline recommendation sequence diagram. 38

4.4 Hyperparameter recommendation sequence diagram. 39

4.5 Activity diagram. 40

5.1 Frequencies of classification & regression algorithms in metalearning database. . 45
5.2 Frequencies of usage of encoding & scaling algorithms in metalearning database. 46
5.3 Format for storing pipelines in the database. 47
5.4 Machine learning editor web application. 48
5.5 Entering information about learning task for recommendation. 49
5.6 Graphical representation of the machine learning pipeline. 49
5.7 Editing hyperparameters of a pipeline block graphically. 50
5.8 Project planning. 58

List of Tables

2.1 Classification datasets used in benchmark. 22

2.2 Benchmark on classification datasets. 22

2.3 Regression datasets used in benchmark. 23

2.4 Benchmark on regression datasets. 23

3.1 Description of the recommendation of a machine learning workflow use case. . . 28

3.2 Description of the automated hyperparameter recommendation use case. 28

4.1 Overview of the computed metafeatures. 35

5.1 Hardware environment. 42

5.2 Software environment. 43

5.3 Technological choices. 43

5.4 Classification algorithms: name, source and number of hyperparameters. 44

5.5 Regression algorithms: name, source and number of hyperparameters. 45

5.6 Data processing algorithms: name, role, source and number of hyperparameters. 46

5.7 Classification datasets: Name, Number of samples, of features (categorical and numerical), of classes and majority and minority class ratios. 52

5.8 Benchmark on Heart Disease UCI dataset. 52

5.9 Benchmark on Airline Passenger Satisfaction dataset. 53

5.10 Benchmark on Pulsar Stars dataset. 53

5.11 Benchmark on Online Shopper's Intention dataset. 54

5.12 Benchmark on Forest Cover Type dataset. 54
5.13 Benchmark on Profitable Customer Segments dataset. 55
5.14 Regression datasets: Name, Number of samples and features (categorical and numerical). 56
5.15 Benchmark on Green Manufacturing dataset. 56
5.16 Benchmark on Bike Sharing dataset. 57
5.17 Benchmark on House Pricing dataset. 57
5.18 Benchmark on Russian Housing Market dataset. 57

Acronyms

AI	Artificial Intelligence.
API	Application Programming Interface.
AutoML	Automated Machine Learning.
CASH	Combined Algorithm Selection and Hyperparameter optimization.
DAG	Directed Acyclic Graph.
HPO	Hyperparameter Optimization.
HTTP	Hypertext Transfer Protocol.
JSON	JavaScript Object Notation.
MAE	Mean Absolute Error.
ML	Machine Learning.
MSE	Mean Squared Error.
RMSE	Root Mean Squared Error.
UML	Unified Modeling Language.

General Introduction

Over the past decade, Machine Learning, an Artificial Intelligence sub-field where computer programs become smarter as they process more data, has penetrated its roots into a wide spectrum of domains and industries, from predictive analytics, to medical diagnosis and personalized recommendations, hoping to leverage data to drive their businesses.

This huge surge, has led to an increasing demand for data scientists positions. According to job recruiting platform Glassdoor, data scientist has been named the best job in America in 2019 in terms of salary, job satisfaction and openings [URL4]. However, with the advent of big data and the quantity of data being generated and consumed by companies, the job supply is not matching the demand of the industry. This has led researchers and professionals to look at ways to automate the data scientists job, with the use of artificial intelligence techniques.

To put it simply, Automated Machine Learning (AutoML) is a sub-field of artificial intelligence that aims to provide, from a labelled dataset as input, an optimized machine learning solution. While the AutoML term dates back to 2014 [URL3], a lot of skepticism has surrounded AutoML, as many researchers question whether AutoML solutions can strike the perfect balance between efficiency and performance.

Indeed, AutoML involves searching in a large combinations set of data processing & machine learning algorithms, with their respective parameters to build a solution, hence the need to find intelligent ways to find the right solution in the search space to avoid combinatorial optimization. Big companies like Amazon, Google and Microsoft are using their massive computational and human resources to solve this issue.

In this contest, we will explore through this internship different AutoML techniques to build a solution that will later be integrated within a code-less machine learning workflow designer web application. We will particularly be exploring Meta-Learning, an AutoML technique which aims to perform machine learning on machine learning experiments to find a solution to the AutoML problem.

This report describes the work achieved during this internship. It is composed of five chapters. The first chapter is a general presentation of the host company & project. The second chapter will focus on a preliminary study of the different AutoML techniques, as well as an analysis of competitive solutions. The third chapter will focus on the requirements specifications and analysis. The fourth chapter gives an overview of the developed AutoML solution architecture. The final chapter will present the development environment, the achieved work as well as benchmarks of our solution with the competition, followed by a general conclusion.

Chapter 1

General Context

Introduction

In this chapter, we will present an overview the internship from a high level. It will be mainly composed of two sections. In the first, we will present the context of the project, in both its academic and professional setting. In the second, we will present the internship projects challenges and objectives.

1.1 Academic Context

This project was elaborated in the context of a graduation internship, the final step for obtaining the National Computer Science Engineering Degree from the National School of Computer Sciences in Tunisia (ENSI). This internship was conducted over a five-month span, from February 3rd to June 30th, during which we integrated the software development team at Integration Objects.

1.2 Professional Context

In this section, we will present Integration Objects, the hosting company which proposed this subject.

1.2.1 Presentation of the Hosting Company

1.2.1.1 Overview

Integration Objects is a software development company founded in 2002, headquartered in Tunisia with sales representatives in Texas and Genoa. In the past decade, it has rapidly emerged to become a leading systems provider and solution integrator in such fields as process automation, command & control centers, operational intelligence, industrial internet of things, cyber security, big data analytics for enterprises, process, power and utilities industries, defense and governmental institutions.

1.2.1.2 Services

Integration Objects offers several services that are characterized in two main sections:

Operational Intelligence: The core objective of this product category is to empower operations, to increase production uptime and to maximize asset availability, normal operations and safety. The product suite includes applications for performance monitoring using key performance indicators, abnormal conditions management, predictive analytics, asset management and decision support. These applications are built on top of Integration Objects' platform, **KnowledgeNet Analytics and Smart Equipment**.

Standard Industry Connectivity & Cyber Security: This product category includes more than 40 OPC based software products enabling users to integrate their systems using industry communication standards. These products enable the access to process data, alarm and event messages, historical data. This data is collected from sensors or PLC.

1.2.2 Staff Organization

Integration Objects teams of engineers are its most important asset. These engineers have different, yet, complementary skills. Some of these disciplines are automation, software development, quality management, industrial and chemical process engineering. The figure 1.1

describes the staff organization of Integration Objects. In this figure we highlighted our current position within the company.

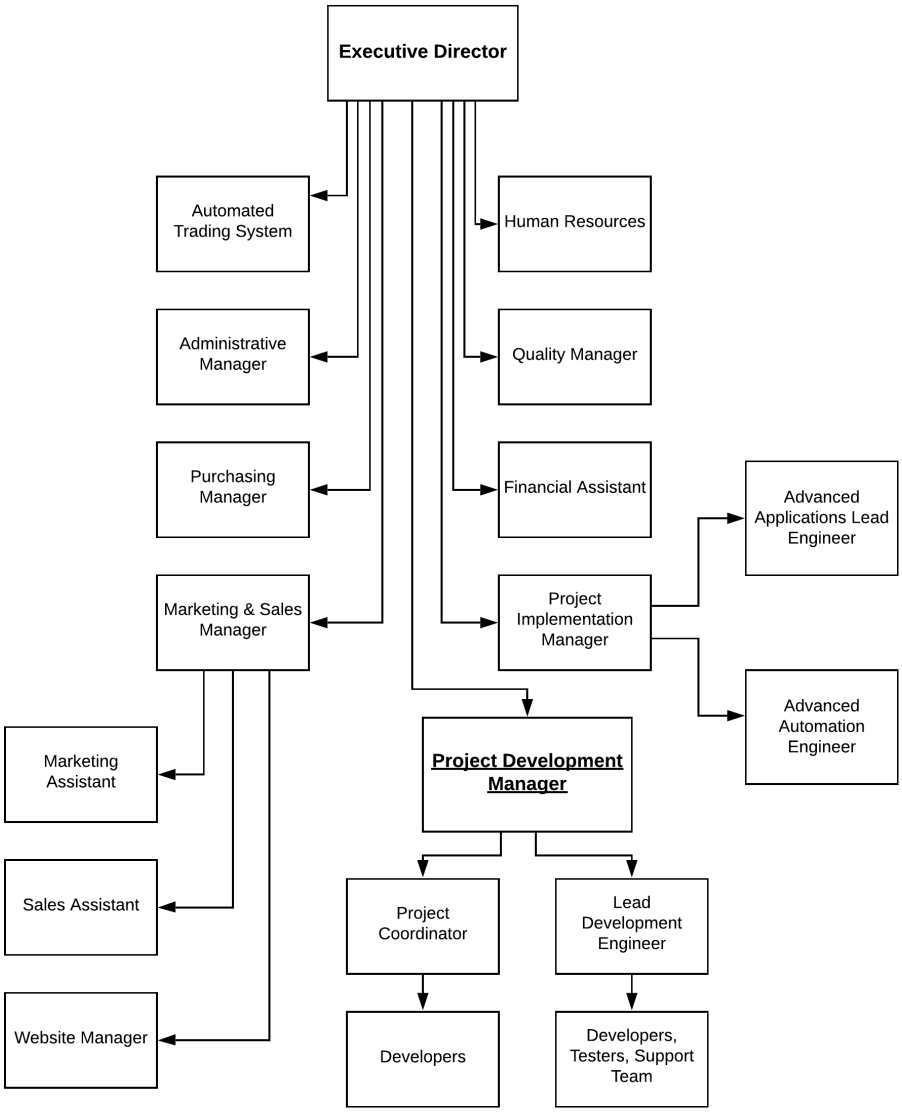


Figure 1.1: Staff organization.

1.2.3 Work Methodology

To enhance product quality and accelerate the process of software development, Integration Objects implemented, based on the ISO 9001 process approach 2008 version, a methodology helping engineers in the design and development of software. It is similar to the V-model which represents the relationships between each phase of the development life-cycle and its as-

sociated phase of testing. During the internship, this methodology was adopted. It advocates a validation phase in between main stages and recommends the use of prototypes throughout the iterative projects life cycle.

The figure 1.2 shows the aspect of this methodology followed by a short description of each phase, via a flowchart diagram, a type of activity diagram that represents a workflow or process.

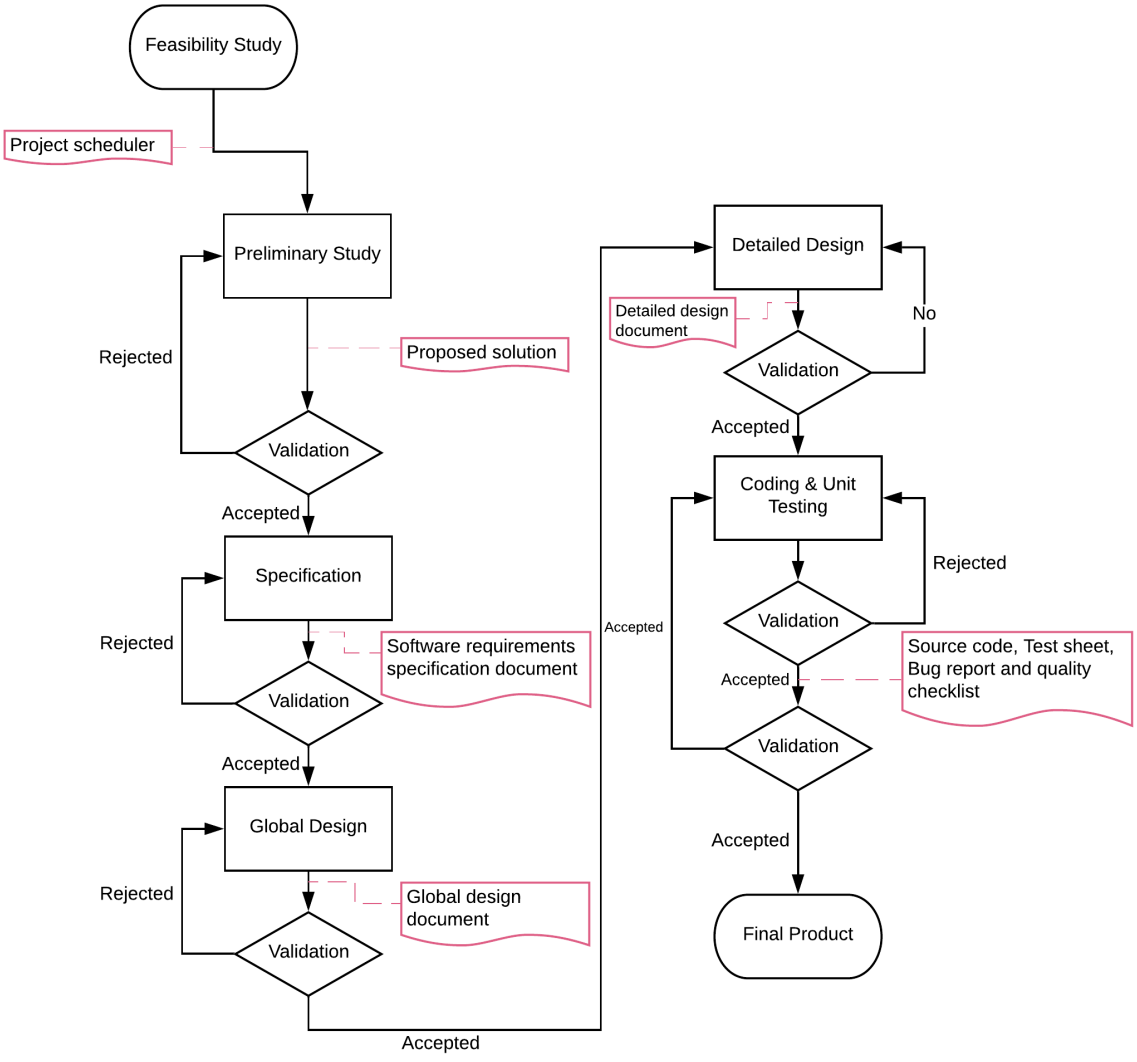


Figure 1.2: Work methodology overview.

The phases of this development processes are:

- **Feasibility Study:** It is the first stage of the ISO process through which a study is provided to determine and to document the projects viability.
  Stage Output: Project Schedule.
- **Preliminary Study:** It represents an initial exploration of existing and possible solutions for the issue addressed.
  Stage Output: Study of existing solutions, proposed solution.
- **Specification:** This phase describes the clients needs and determines the system features.
  Stage Output: Software requirements specification document.
- **Global Design:** The fourth phase describes the architecture of the product and its main parts.
  Stage Output: Global design document.
- **Detailed Design:** In this phase, we will identify how to implement efficiently the requested product in order to meet the needs described in the specification.
  Stage Output: Detailed design document.
- **Coding and unit testing:** Developers have to implement the entire solution to validate functional features.
  Stage Output: Code source, Test sheet, bug report, Source code quality checklist.

1.3 Project Context

In this section, we will present a high-level overview of the project challenges and objectives.

1.3.1 Project Challenges

Nowadays, lots of machine learning algorithms have been invented, giving scientists and engineers a rich toolbox of algorithms to choose from. However, the choice is becoming a difficulty for data scientists, as each algorithm has its own unique set of parameters that require proper tuning depending on the use case.

Moreover, the task of finding a well-performing end-to-end pipeline which can satisfy the machine learning problem is a tedious task that requires a certain level of expertise.

Also, the data fed to each of the algorithms has to respect a given data structure for compute optimizations. With these constraints, data scientists have to spend hours writing code, training, testing, and evaluating machine learning models in order to find solutions that satisfy their use case requirements.

1.3.2 Project Objectives

In order to minimize the time and cost of developing machine learning models, Integration Objects aims to provide a code-less editor for designing and executing machine learning pipelines. The editor should be web based providing users the ability to graphically design and execute machine learning pipelines.

On top of the basic features of the editor, the system should include an intelligent recommendation sub-system to reduce the cost, in effort and in time, of developing machine learning pipelines. The automated machine learning (AutoML) system should recommend end-to-end machine learning pipelines, along with their hyper-parameters.

Conclusion

In this project, we provided a high-level overview of the project. We started by presenting the academic and professional context of the project, we presented the objectives we aim to accomplish during this internship. In the next chapter will be dedicated to a preliminary study where we will introduce theoretical concepts needed to understand the rest of the document, and a study of existing AutoML solutions.

Chapter 2

Preliminary Study

Introduction

In this chapter, we will present a preliminary study of the project to ensure a thorough understanding of the project. In the first section, we will present the key theoretical concepts this project is built upon. The second section will be dedicated to a comparison of existing solutions.

2.1 State of the Art

In order to successfully develop our project, a state of art study should be made. This section details the main theoretical concepts needed to understand during the project's realization.

2.1.1 Machine Learning Pipelines

In computer science, a pipeline is defined as a set of processing steps connected in series, where the output of one element is the input of the next element.

Machine learning pipelines represent pipelines destined to solve machine learning workflows, such as classification or regression. Machine learning pipelines are composed of data processing algorithms (data cleaning, feature selection, feature engineering, etc), with an esti-

mator as a final step. Each block is also composed of its corresponding hyperparameters.

Machine learning pipelines can be represented as a directed acyclic graph (DAG), where each step is the input of the next step, or can be more complex, where the input of a privitime is the union/concatenation of several blocks, or where sub-pipelines are only applied to a specific type of feature set (numeric or categorical) as seen in the figure 2.1 below [7].

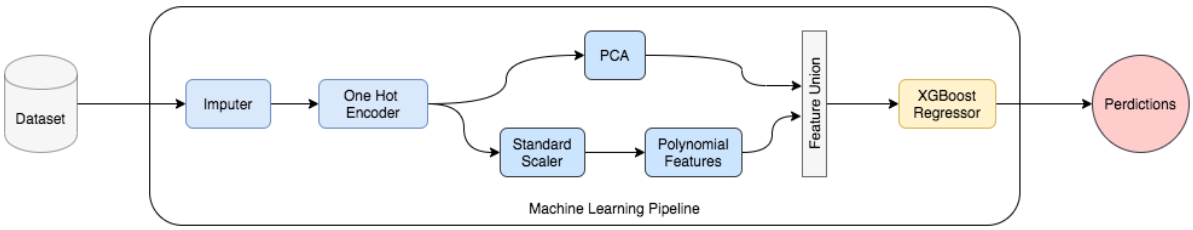


Figure 2.1: Example of a task-specific machine learning pipeline.

2.1.1.1 Advantages of Machine Learning Pipelines

The main advantages of machine learning pipelines are [URL11]:

- **Flexibility:** data mining & machine learning primitives are easily replaceable in a machine learning pipeline.
- **Reusability:** machine learning pipelines are made to be easily reusable across similar ML tasks.
- **Efficiency:** ML pipelines simplify the automation of ML workflows.

2.1.1.2 Machine Learning Pipeline Steps

Machine learning pipelines generally contain patterns of blocks designed for a particular task in order to ensure that a dataset as input can be processed later by a machine learning algorithm. These steps are illustrated in the figure 2.2:

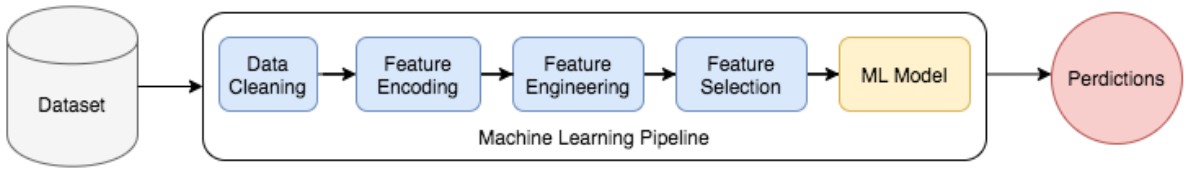


Figure 2.2: Template of a machine learning pipeline. Differences exist regarding the data processing steps.

- **Data Cleaning:** consists in detecting, correcting and/or removing inconsistent, erroneous or missing data from a dataset. In the context of automation, this usually means imputing missing values from the dataset with a certain strategy, depending on the machine learning task and the feature type [URL15].

- **Data encoding:** is the process of turning categorical data into usable, machine-learning ready, numerical data, this step ensures that categorical features can be processed by the model, while keeping a logical consistency between the original data and the encoded features.

- **Feature engineering:** is the process of transforming the original data into a form which is easier to interpret by the machine learning model. It also means generating new features that can contain better patterns to identify for the learning algorithm. Some ways to perform feature engineering include scaling, normalization, feature discretization, etc. [URL6]

- **Feature selection:** consists in reducing the number of features for the predictive model in order to improve performance and reduce computational complexity. Feature selection techniques involve evaluating the dependencies between input features and the target, and keeping the features with the strongest relationships [URL8].

- **Machine learning model:** The final step in every machine learning pipeline, it consists of a learning algorithm that takes as input the data generated by the previous pipeline steps, makes a prediction on that input data and then returns it as a response. We will focus on this project on supervised learning models, for both regression and classification problems.

2.1.2 Automated Machine Learning (AutoML)

Automated machine learning is a recent field of interest in artificial intelligence that aims to automate the building of machine learning models/workflows to solve data science tasks.

The AutoML problem is as follows: given a dataset, and a machine learning task, the goal of an AutoML system is to solve the learning task of the given dataset while optimizing the performance and respecting specified constraints (like time) [11]. The figure 2.3 illustrates the general architecture of AutoML systems.

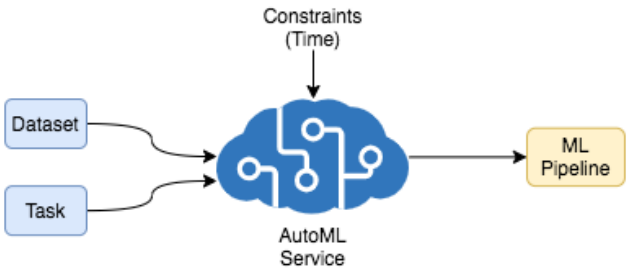


Figure 2.3: General architecture of AutoML systems.

This advancement in AI reduces the need of machine learning experts, and hence democratizes the field of machine learning to people with little data science or machine learning expertise, as well as improve the productivity of data scientists. Most major IT companies (Microsoft, Google, Amazon, etc) are investing money and resources in building efficient AutoML systems and researchers continue to push the field forward.

2.1.2.1 Techniques of AutoML

Combined Algorithm Selection and Hyperparameter optimization (CASH): The first attempt at AutoML, before the term AutoML existed, came in 2013 with solutions such as **Auto-WEKA (2013)** [9]. Combining both the learning algorithm search as well as its parameters is due to the fact that no learning algorithm performs better than others on all datasets, and hyperparameters can heavily impact model performance [5]. The first iterations of such solutions relied on classic grid and random search solutions, but due to the high-dimensionality of the CASH problem, more efficient search algorithms were required to solve the optimization objective, like Bayesian Optimization.

Bayesian Optimization: In order to take the human out of the optimization process, more efficient optimization needed to be explored to achieve the goals of AutoML.

Bayesian optimization is a black-box optimization technique that uses past evaluations to intelligently choose the algorithm and set of hyperparameters to evaluate next, as demonstated in the figure 2.4. This informed hyperparameter search technique requires less iterations than uninformed search algorithms to get to an optimal set of hyperparameters and algorithms [URL10].

This technique has been employed by researchers in solutions such as **Auto-WEKA (2013)** [9] and **Auto-Sklearn (2015)** [3], and even in production by **Microsoft** in their Azure cloud platform and desktop work environments.

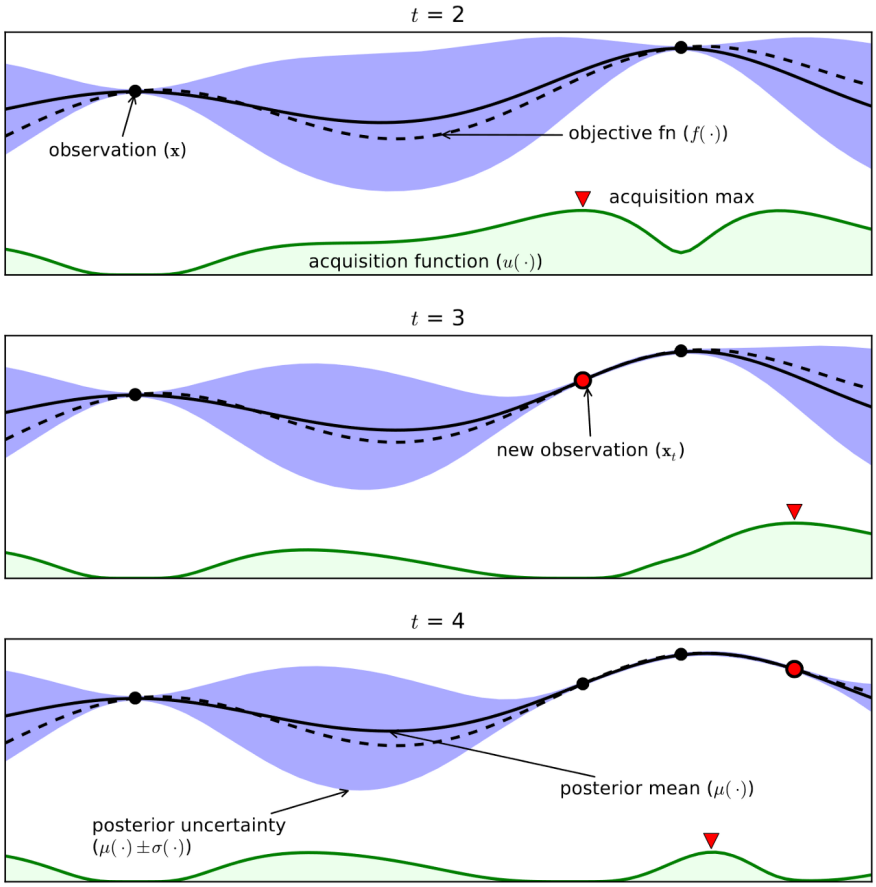


Figure 2.4: Iterations of Bayesian optimization. [URL13]

Evolutionary Algorithms: Starting from 2015, researchers started looking at evolutionary algorithms techniques, particularly genetic programming, for generating machine learning pipelines, with tools such as **TPOT (2015)** [6].

Genetic programming is an artificial intelligence method for solving optimization problems, inspired by the theory of biological evolution. From an initial set of random solutions, a new family of solutions are derived via mutation operators and crossovers to generate a new family of solutions [1]. This flow is illustrated in figure 2.5

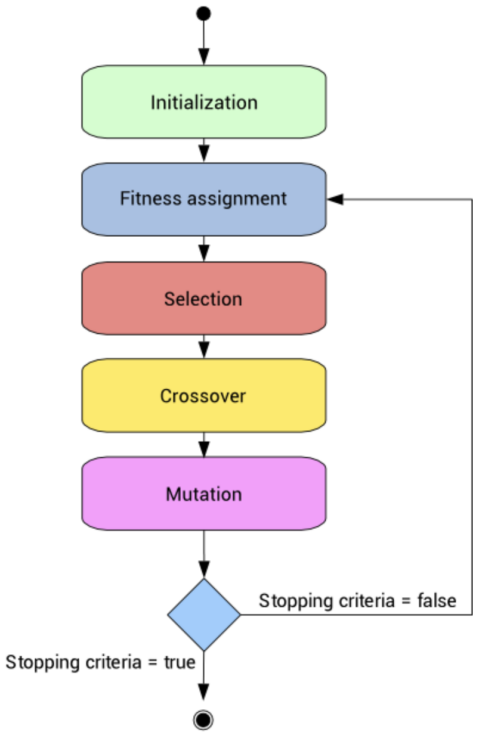


Figure 2.5: Flowchart of genetic algorithms. [URL7]

In the context of AutoML, researchers started to adapt the optimization technique for pipeline generation. Operations in a machine learning pipeline are considered as genetic programming primitives, and pipelines as genetic programming trees. Mutation operators consist in changing parameters in a block, to changing a block in a pipeline entirely. Crossovers between pipelines allow for the generation of a new family of pipelines.

In spite of the efficiency of this technique, intensive resource & high execution time remains the most blocking handicaps.

Reinforcement Learning: Reinforcement learning is a branch of machine learning where a computer program, or agent, learn not from data but from observations by interacting with its environment to achieve a goal [8].

Reinforcement learning has been heavily used in robotics and gaming, as its very nature is similar to problems in that domain. But reinforcement learning has also been used in a variety of domains like finance and trading, and now in AutoML.

Since 2018, the AutoML problem found applications in reinforced machine learning areas and many works have been developed such as **AlphaD3M (2018)** [2] and **DeepLine (2019)** [4]. The agent in this case interactively builds a machine learning pipeline by selecting from a set of actions such as insertions, deletions or replacements of pipeline blocks and parameters, with the reward defined by the performance of the newly generated pipeline on that particular dataset.

Ensemble Learning: Ensemble learning is a machine learning technique where multiple machine learning models, or "weak learners", are trained on the same dataset and combined in an ensemble model, or strong learner to get better results. AutoML solutions from **H2O** and **Microsoft** started to use ensemble methods, mainly stacking and voting, to achieve better performance.

Stacking models, as illustrated in figure 2.6 below, use the predictions of the weak learners as features of the stacking model, or meta-learner, to generate a prediction. This reduces the bias of single models, and the resulting model hence generalizes better.
Voting models learns the right weights to ponder the predictions of the weak learner in order to offer a final prediction.

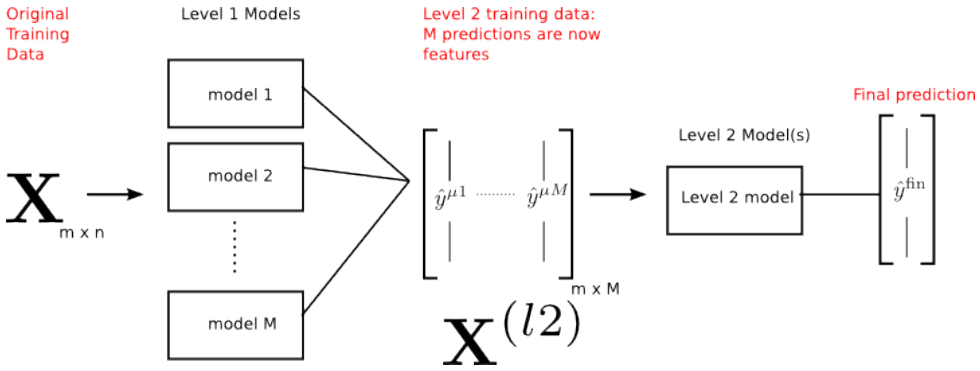


Figure 2.6: Stacked ensembles illustrated. [URL14]

H2O uses stacking on a predefined list of models built during the specified time constraint, meaning that the best solution can only be generated at the end of the optimization process.
Microsoft generates stacking and voting ensembles in its Azure suite to offer possibly better models from the pipelines generated during the optimization process.

This method, while effective, is heavily dependent on the quality of the weak learners, and generated models can be too complex for production solutions.

2.1.2.2 Goals of AutoML

The three main pillars that any AutoML solution strives to achieve are:

- **Good performance:** an AutoML solution should provide good generalization performance across various datasets and learning tasks.
- **Less assistance from humans:** AutoML should take the human out of the machine learning task as much as possible.
- **High computational efficiency:** The AutoML solution should run without being too process-intensive, in the shortest time constraints possible.

2.1.2.3 Challenges of AutoML

The main challenge of AutoML is efficiency, as most solutions are building and evaluating several machine learning models and pipelines to find the best solution, this makes the task of building a machine learning solution using AutoML both time and resource intensive, and can result in poor performance if not given the necessary computational resources.

2.1.3 Metalearning

Metalearning, or learning to learn, is an AutoML method that consists in observing how different machine learning solutions perform on several learning tasks, extracting metadata from those experiments and learning from those experiences to learn solutions to new tasks much faster than otherwise. Essentially, it means applying machine learning on machine learning experiments.

When data scientists approach a machine learning task, they use their prior knowledge to get a quick and efficient solution to the task. Meaning that they apply similar data preprocessing techniques used on similar prior datasets, and apply machine learning models they already experimented with on similar tasks. And with experience, the amount of trial and error is significantly reduced.

The challenge, in context of machine learning, is to find ways to transform this human learning instinct into data suitable for learning tasks, and formulate this into a machine learning problem [10].

Indeed, we need to collect metadata on both datasets and reference solutions, meaning the blocks representing the machine learning pipeline with their hyperparameters, as well as information to describe the datasets. We will also need to build learning algorithms to exploit this knowledge to solve AutoML tasks. The approach of metalearning for AutoML is illustrated in the figure 2.7.

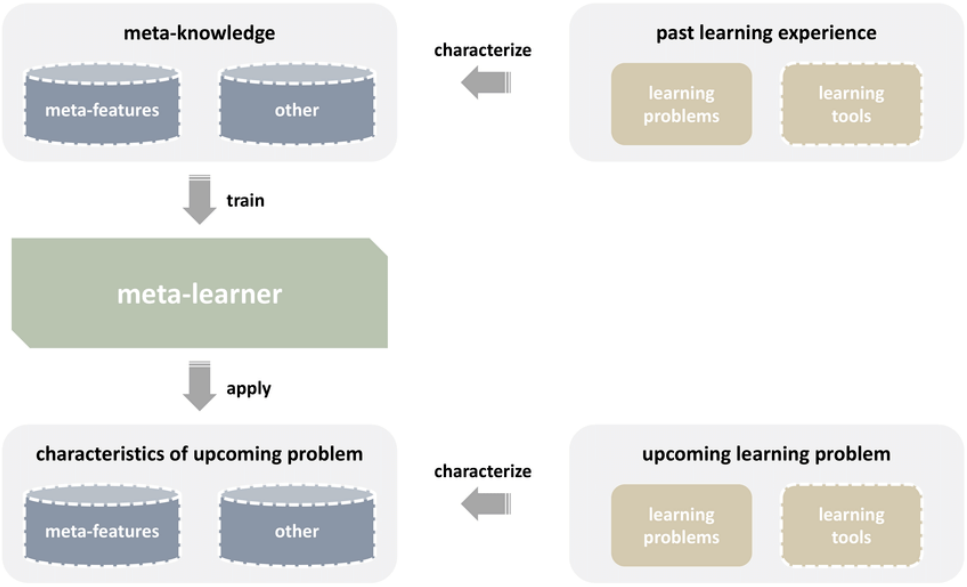
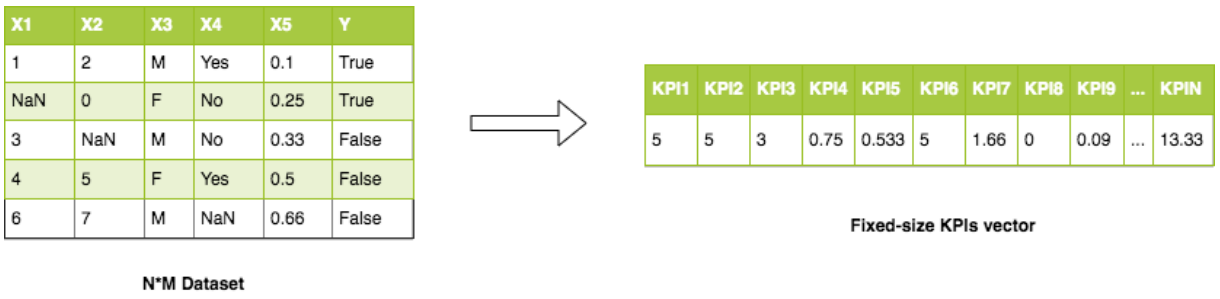


Figure 2.7: General process of metalearning. [11]

2.1.3.1 Metafeatures

Meta-learning based solutions relies on extracting metadata from learning tasks. The most common format to represent these metadata is meta-features, containing characterizations of the learning task. Each task is represented by a fixed-size vector of meta-features. The type of task (i.e. classification, regression, etc) defines the size of the vector. This transformation will allow to calculate dataset similarities which will lead us to pipeline selection based on metafeature-vector comparison. The figure 2.8 demonstrates how metafeatures transform a dataset to a fixed-size vector.



X1	X2	X3	X4	X5	Y
1	2	M	Yes	0.1	True
NaN	0	F	No	0.25	True
3	NaN	M	No	0.33	False
4	5	F	Yes	0.5	False
6	7	M	NaN	0.66	False

N*M Dataset

KPI1	KPI2	KPI3	KPI4	KPI5	KPI6	KPI7	KPI8	KPI9	...	KPIN
5	5	3	0.75	0.533	5	1.66	0	0.09	...	13.33

Fixed-size KPIs vector

Figure 2.8: Transforming dataset into fixed-size metafeatures vector.

Metafeature categories: Meta-features are used to characterize datasets, by extracting relevant information that will be later used for comparison. These characteristics can be divided into the following categories:

- **Descriptive metafeatures:** Used to describe common characteristics of datasets, like the number of instances and features, the relationship between categorical and numerical features, information about null values, dataset dimensionality, and information about target features, for both classification and regression, to name a few.
- **Statistical metafeatures:** contain standard statistical measures to describe the numerical properties of a distribution of data
- **Info-theoretical metafeatures:** describe discrete (categorical) attributes and their relationship with the classes in classification tasks.
- **Correlation metafeatures:** Used to describe correlations between numerical features and the target variable in regression tasks.
- **Landmarking metafeatures:** consists in extracting performance metrics from simple, fast learners, i.e. machine learning models with a train time complexity lower than $O(nm)$, n being the number of observations and m the number of features in the dataset. These can be used to identify learning tasks that perform similarly on a specific learner. Such low complexity learners are: Gaussian Naive Bayes, Latent Dirichlet allocation, Decision trees or Extra trees with shallow heights (≤ 2) [URL5].

Exploiting metafeatures: Meta-feature vectors can be used to compare between similar learning tasks, using for example Euclidean distance, so we can transfer the known solution of a reference dataset to the new learning task. However, more robust ways to study similitude are needed than classic distance metrics, since these can be very sensitive to the variations of a single feature in the meta-features vector.

2.1.3.2 Meta-learners

Metafeatures can be exploited as training data for models that can predict performance of a machine learning pipeline on a learning task. This model can be later used to rank a set of

solutions to a machine learning task based on their predicted performance. This means that a machine learning pipeline, with its corresponding hyperparameters, in a database of pipelines, can be chosen as a potential optimal solution without ever executing it.

This does however, require keeping track of the performance metrics of several machine learning runs, as well as figure out the right combination of metafeatures to use as learning features from both reference solutions and new learning tasks. Concise testing is also required to ensure that meta-learners can achieve a satisfying performance on several learning tasks.

2.2 Study of the existing

In this section, we will perform a survey and benchmark on AutoML solutions. In the first part, we will discuss existing AutoML frameworks and how they approach the optimization problem. In the second part, we will provide a benchmark and analysis of their performance on reference datasets.

2.2.1 Existing AutoML frameworks

This section presents the AutoML frameworks used for benchmark. These solutions are able to build ML pipelines or models using distinct optimization techniques.

2.2.1.1 Auto-sklearn

Auto-sklearn [3] is a tool for constructing machine learning pipelines. The pipelines all have a fixed structure: at first, a fixed set of data cleaning steps, including optional categorical encoding, imputation, removing variables with low variance and optional scaling, is executed. Next, an optional pre-processing and mandatory machine learning model are selected and tuned via sequential model-based algorithm configuration (SMAC), which is a tool based on Bayesian optimization in combination with a racing mechanism on the instances to efficiently decide which of two configurations performs better. Auto-sklearn also uses meta-learning to warm start the initialization process, and optionally can generate an ensemble of the best performing pipelines.

Auto-sklearn is built in Python and uses Sklearn for its data processing and machine learning algorithms.

2.2.1.2 Tree-based Pipeline Optimizer (TPOT)

TPOT [6] is a framework for building and tuning machine learning pipelines. It uses genetic programming to build pipelines and to select an algorithm in each pipeline stage. Since pipelines recommended by TPOT aren't defined by a specific structure, and to avoid recommending complex pipelines prone to over-fitting, TPOT selected from a Pareto front using a multi-objective selection strategy between high performance and low complexity. At the end of the execution, TPOT returns the single best performing pipeline.
TPOT is built in Python, and uses Scikit-learn as its back-end for data processing and machine learning algorithms.

2.2.1.3 H2O AutoML

H2O is a machine learning framework built to assist data scientists in every aspect of building distributed machine learning solution. The H2O framework contains an H2O AutoML component. H2O AutoML is able to automatically select and tune a classification or regression algorithm without data pre-processing steps. Configurations are generated using random grid search during a specified runtime budget. At the end of the execution, H2O aggregates built models in a stacked ensemble. [URL2]
H2O is, unlike other considered frameworks, built using Java with a Python or R library to communicate with the Java back-end.

2.2.1.4 Microsoft ML.NET AutoML

Microsoft offers AutoML solutions in both cloud and desktop environments. As for their desktop solution, it is a command-line tool capable of generating and tuning machine learning models or pipelines for classification, regression and recommendation.
At the end of the trial run, where the user can specify the pipeline generation duration, a leaderboard of the top-performing solutions is presented to the user. This command line returns the best-performing model, as well as C# code for running or training the model. [URL1]

2.2.2 Benchmark of AutoML frameworks

In this section, we present a comparison between most prominent AutoML framework. All frameworks have been set at default settings, and to run during one hour. All results are computed after 5-fold cross validation.

2.2.2.1 Classification benchmarks

Metrics In this classification benchmark, we will use the following performance metrics to compare the AutoML frameworks.

- **Accuracy:** the proportion of true results among the total number of cases examined. Higher is better.
- **F1-Score:** weighted average of the precision and recall. Higher is better.

Datasets In the table 2.1, we present the datasets used in this classification benchmark along with descriptive metrics. Both datasets are binary classification tasks.

Dataset	# Samples	# Features	# Num.	# Cat.	Maj. Class	Min. Class
Breast Cancer Wisconsin	569	30	30	0	0.63	0.37
Online Shopper's Intention	12330	17	10	3	0.8453	0.1547

Table 2.1: Classification datasets used in benchmark.

Results The table 2.2 summarizes the comparison results of the classification benchmark, where H2O and Microsoft AutoML frameworks offered better performance than rivals.

Dataset	Breast Cancer Wisconsin		Online Shopper's Intention	
AutoML Tool	**Accuracy**	**F1-Score**	**Accuracy**	**F1-Score**
Auto-Sklearn	0.951	0.96	0.9078	0.6432
TPOT	0.979	0.983	0.897	0.633
H2O AutoML	0.9444	0.957	**0.9095**	**0.675**
Microsoft ML.NET AutoML	**1.0**	**1.0**	0.9078	0.6725

Table 2.2: Benchmark on classification datasets.

2.2.2.2 Regression benchmarks

Metrics During this regression benchmark, we will use the following performance metrics for comparison:

- **Mean Absolute Error (MAE):** the absolute difference between the target value and the value predicted by the regression model. Lower is better.
- **Mean Squared Error (MSE):** the average of the squared difference between the target value and the value predicted by the model. Lower is better.
- **R-Squared:** a metric that compares the current model with a constant baseline and tells us how much our model is better. The constant baseline is chosen by taking the mean of the data. Higher is better.

Datasets The table 2.3 below presents the datasets used in the regression comparison benchmark.

Dataset	# Samples	# Features	# Numeric	# Categorical
Boston Housing	506	14	13	1
Used Cars Catalog	12330	29	6	23

Table 2.3: Regression datasets used in benchmark.

Results The table 2.4 summarizes the comparison results of the regression benchmark. In both learning tasks, H2O AutoML performed better than rivals.

Dataset	Boston Housing			Used Cars Catalog		
AutoML Tool	**MSE**	**MAE**	**R-Squared**	**MSE**	**MAE**	**R-Squared**
Auto-Sklearn	12.917	2.56	0.87	3300648.93	1003.75	0.922
TPOT	14.82	2.79	0.842	3851345.42	1125.91	0.906
H2O AutoML	**6.111**	**1.8**	**0.928**	**2349122.27**	**890.47**	**0.943**
Microsoft ML.NET AutoML	14.49	2.71	0.8504	2591337.98	928.52	0.941

Table 2.4: Benchmark on regression datasets.

2.3 Proposed Solution

During this study of the existing, we observed that most AutoML solutions rely on an iterative process, where the solver starts with a random model or pipeline and iteratively improves it to build a solution. Metalearning has only so far been used as a way to warm start the initialization process. To provide a solution that is both resource efficient and with strong performance, we will build an AutoML system that works in two phases:

- Use advanced metalearning models to propose a solution from a curated set of pipelines, without ever executing them.
- Use state-of-the art Bayesian optimization techniques to run fully automated hyperparameter optimization jobs.

Conclusion

In this chapter, we explored the state of the art of our project. We started by defining theoretical concepts used during the history of AutoML, as well as their relative shortcomings. We then studied and bench-marked the existing solutions, their functionalities, their limits and we ended up proposing our solution. The next chapter is dedicated to outlining and specifying the project's requirements and objectives.

Chapter 3

Specification & Requirements

Introduction

To successfully complete this project, we need to have a clear and precise understanding of all its features. This implies a thorough study and analysis of all its aspects and components, which will be detailed in this chapter.
This chapter will do so by highlighting the objectives and main goals that our solution must reach, and will be presenting the functional and nonfunctional requirements of the project, all of which will be detailed using adequate diagrams and use cases based on the UML formalism.

3.1 Requirement Analysis

In this part, we will focus on the analysis of our requirements. We will start by identifying the actors of the system and then we will quote the functional and non-functional requirements of our application.

3.1.1 Identifying Actors

This project is aimed at data scientists, with different levels of expertise. Novice users can use the code-less interface and the AutoML recommendation system to generate and execute machine learning workflows for their dataset, without worrying about code. Expert users will also benefit by the added automation of the project, by focusing on domain specific treatments,

and allowing the AutoML system to perform most of the repetitive data science task. They can also further refine the results obtained by the recommendation system based on their expertise.

3.1.2 Functional requirements

In this section, we present the functional requirements our solution should fulfill. In our case, our AutoML recommendation system should work in two modes:

- recommend end-to-end ML pipelines, including data processing & ML blocks and their respective parameters.
- tune hyperparameters of an ML pipeline, given a time constraint.

Furthermore, the recommendation system should exist within a machine learning pipeline designer web application, which should allow the user to:

- build a machine learning workflow by arranging and connecting blocks that form a machine learning pipeline.
- edit recommended or built machine learning workflows.
- run a defined machine learning workflow.

3.1.3 Non Functional Requirements

In this section we identify the main non-functional requirements to consider while designing and developing the application:

- **Performance:** The provided system should achieve comparable performance compared to existing solutions.
- **Efficiency:** The end-to-end recommendation system should propose a solution to the learning task in a worst case time complexity of $O(nm)$, n and m denoting the number of instances and features respectively of the given dataset.
- **Usability:** The recommender system should work with the minimal amount of human assistance. Moreover, the platform should be usable by both experimented data scientists and novice users.

- **Interoperability:** The platform should be easy to integrate with other solutions provided by the company.
- **Maintainability:** The project should be well documented and the source code legible for future software revisions.

3.2 Requirements Specification

In this section, we present the requirements of the AutoML recommendation system with a general use case diagram and activity diagrams conforming to the unified modeling language.

3.2.1 General use case diagram

In the figure 3.1 below, we present the general use case diagram of the AutoML recommendation system.

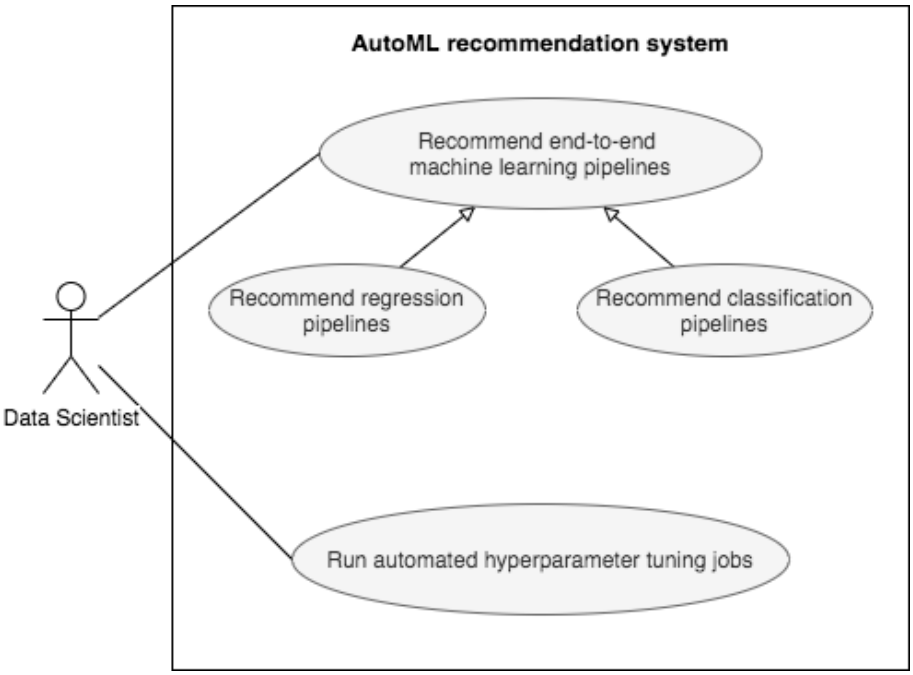


Figure 3.1: General use case diagram.

The table 3.1 provides a description of the main use case which consists in end-to-end machine learning pipeline recommendation.

Use Case Name	**Recommend end-to-end machine learning pipeline**
Brief Description	The user can decide to get a recommendation of an entire machine learning workflow for his learning task, including data processing learning algorithms and their hyperparameters
Actors	User (Data Scientist)
Pre-Conditions	User specifies dataset path or URL, target feature name and learning task type (classification or regression)
Post-Conditions	Pipeline returned to the web application and rendered graphically

Table 3.1: Description of the recommendation of a machine learning workflow use case.

The table 3.2 below describes the second main use case which is automated tuning of pipeline hyperparameters.

Use Case Name	**Run automated hyperparameter tuning job**
Brief Description	A hyperparameter tuning job can either be started automatically started after pipeline recommendation or on a pipeline designed by the user.
Actors	User (Data Scientist)
Pre-Conditions	User specifies dataset path or URL, target feature name and learning task type (classification or regression) and maximum running time
Post-Conditions	A list of new hyperparameters that and their values is returned and updated in the application

Table 3.2: Description of the automated hyperparameter recommendation use case.

3.2.2 Sequence diagram

In this section, we present a sequence diagrams for the global end-to-end pipeline recommendation process. The figure 3.2 illustrates this process. We will be using a metalearning-based approach for pipeline recommendation. The user will present relevant data about the learning task, including path or URL of the dataset, target feature name and whether the learning task is a classification or regression task. Then, metafeatures are extracted by the system to use as input data for the metalearner, which will choose the best pipeline based on the extracted metrics, and the knowledge acquired during several learning tasks. If the pipeline esti-

mated performance is lower than a threshold, a hyperparameter tuning job will automatically be launched. Then, the pipeline is rendered graphically by the web application and trained on the dataset. Moreover, the web application will request the user if he wants to run a hyperparameter tuning job.

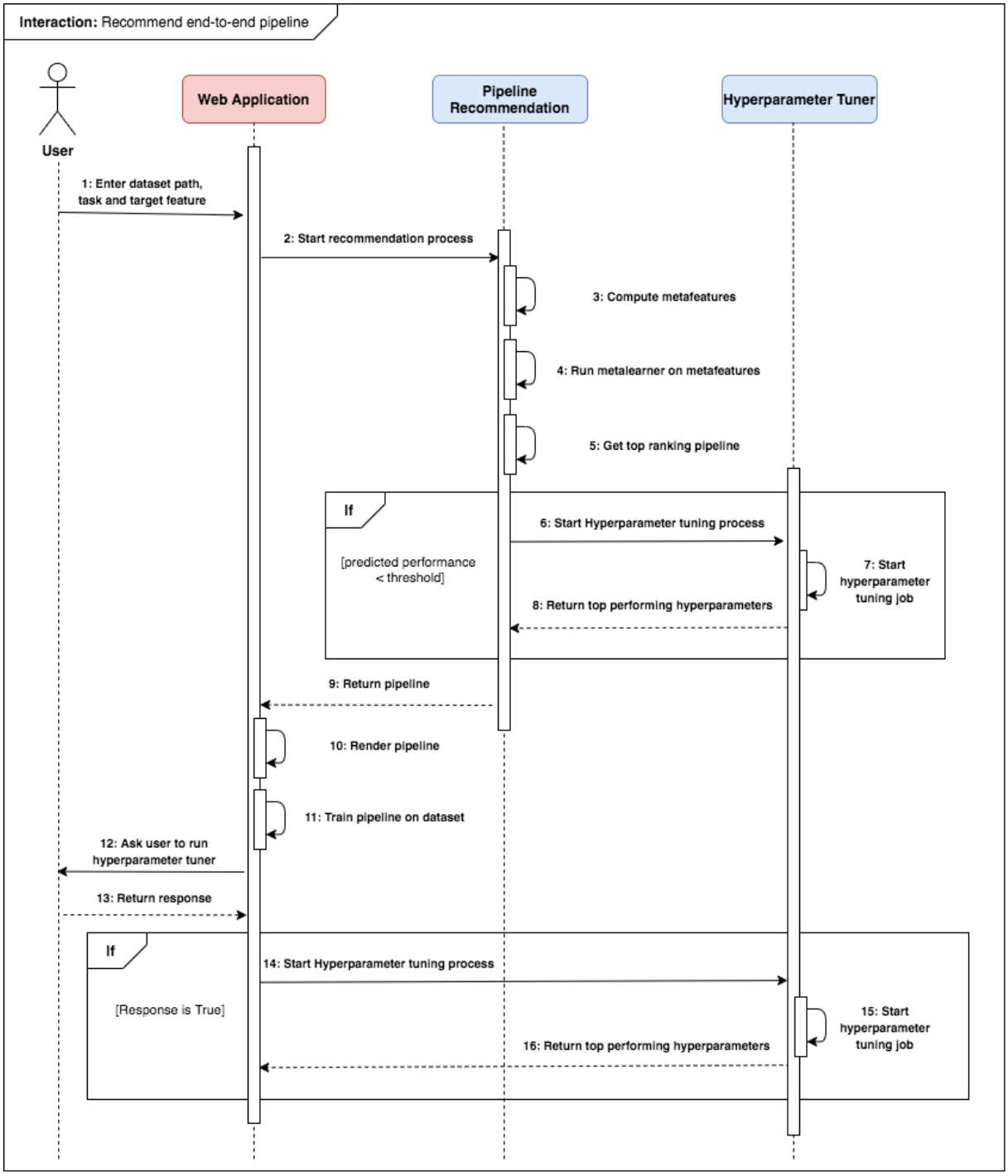


Figure 3.2: Sequence diagram for pipeline recommendation.

Conclusion

The requirements analysis and specification phase allowed us to have a clearer image of the project, as well as the tasks to be fulfilled and the quality standards to be achieved, by analysing the project's functional and non-functional requirements. These requirements will be the foundation for the design of the application, which will be detailed in the following chapter.

Chapter 4

Design

Introduction

Having reviewed the requirements and specifications of the AutoML system, we will now suggest the design chosen during the course of this project. Design is a vital step in the life-cycle of a software project as it has mainly two purposes. The first is the elaboration of detailed models of the system's architecture, and the second is providing a solid ground for the implementation phase. We will present in this chapter the global design of our project, then we will go deeper into certain aspects of the solution via diagrams conform to the unified modeling language (UML).

4.1 Global Design

In this section we will describe the global architecture of the AutoML system, and all the components that ensure proper operation.

4.1.1 General Architecture

The general architecture of the project is described in the figure 4.1 alongside the interaction between the different components. Its main two components (end-to-end pipeline recommendation and hyperparameter optimization) will be detailed in the following subsections.

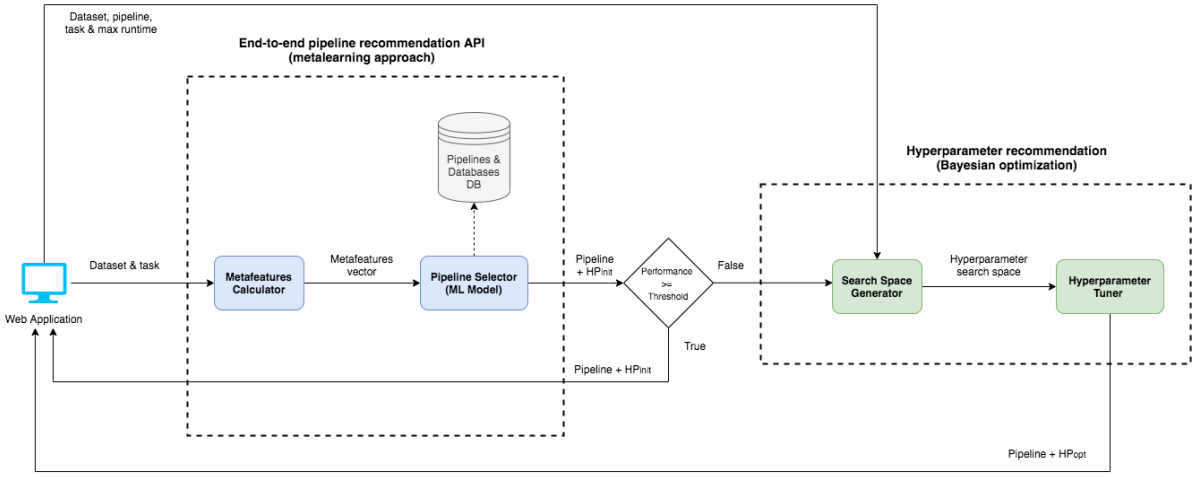


Figure 4.1: Global architecture.

4.1.1.1 End-to-end pipeline recommendation

The end-to-end pipeline recommendation is the main asset of the AutoML project. It can recommend an entire pipeline with its hyperparameters in a worst case time complexity of $O(nm)$, n and m denoting the number of instances and features of the dataset. This means that, depending on the size of the datasets, a solution to the learning task can be recommended in seconds. Its main components are detailed in the following.

Metalearning database Metalearning requires a large amount of metadata collected to ensure strong recommendations. We found an open-source metalearning database named **Niseko** [7] which is *"an open-source large-scale metalearning dataset with more than 5 million pipelines and 50 million pipeline runs on 300 datasets with clear-defined structures"*.
To exploit this database, some changes were made. First, for each reference dataset, we only kept the best performing pipeline. We also made slight changes to the pipeline structure to ensure genericity and to eliminate possible execution errors on different datasets, while making sure these changes don't affect its performance.
In the end, we obtained 215 and 70 classification and regression learning tasks respectively, alongside their reference pipelines and their extracted metafeatures.

Metafeatures calculator The metafeatures calculator allows to transform the input dataset into a format that can be used for learning and comparison. The metafeatures calculator turns an input dataset of any shape into a fixed-size vector of metafeatures from 5 different categories. The learning task defines the size of the vector.
The table 4.1 details the different calculated metafeatures, their variants and whether they are used in classification and/or regression tasks.

Metafeature	Category	C	R	Description	Variants
Nb of instances	Descriptive	✓	✓		log
Nb of features		✓	✓		
Nb of numeric features		✓	✓		
Nb of categorical features		✓	✓		
Dataset dimentionality		✓	✓	$\frac{\text{Nb instances}}{\text{Nb features}}$	log
Inverse dataset dimentionality		✓	✓	$\frac{\text{Nb features}}{\text{Nb instances}}$	
Ratio numeric categorical features		✓	✓	$\frac{\text{Nb numeric features}}{\text{Nb categorical features}}$	
Ratio categorical numeric features		✓	✓	$\frac{\text{Nb categorical features}}{\text{Nb numeric features}}$	
Number of classes		✓	✗		
Class entropy		✓	✗	Class imbalance	
Target class probabilities		✓	✗	Class imbalance	mean, stdev, min, max, median, q1, q3
Target feature statistics		✗	✓		mean, stdev, min, max, median
Numeric feature means	Statistical	✓	✓		mean, stdev
Numeric feature standard deviations		✓	✓		mean, stdev

Numeric feature standard deviations		✓	✓		mean, stdev
Numeric feature variances		✓	✓		mean, stdev
Numeric feature minimums		✓	✓		mean, stdev
Numeric feature maximums		✓	✓		mean, stdev
Numeric feature medians		✓	✓		mean, stdev
Numeric feature skewness		✓	✓		mean, stdev
Numeric features kurtosis		✓	✓		mean, stdev
Numeric features covariances		✗	✓		mean, stdev
Numeric features correlation with target	Correlation	✗	✓		mean, stdev, min, max
Categorical features entropies	Info-theory	✓	✗		mean, stdev
Categorical features mutual information with target		✓	✗		mean, stdev
Gaussian Naive Bayes performance	Landmarking	✓	✗		accuracy, balanced accuracy
Linear Discriminant Analysis performance		✓	✗		accuracy, balanced accuracy
Decision Tree Classifier performance		✓	✗	max depth = 1 and 2	accuracy, balanced accuracy

Extra Trees Classifier performance		✓	✗	max depth = 2, number of estimators = 10	accuracy, balanced accuracy
Decision Tree Regressor performance		✗	✓	max depth = 1 and 2	mean absolute error, mean squared error, r-squared
Extra Trees Regressor performance		✗	✓	max depth = 2, number of estimators = 10	mean absolute error, mean squared error, r-squared

Table 4.1: Overview of the computed metafeatures.

Pipeline selector To exploit the extracted metafeatures, we needed to find a way to compute the relationship between a new learning task and one that is already present in the database. The idea is that if a learning task is similar to one that is stored in the metalearning database, then the existing solution can potentially be a strong candidate for the new machine learning problem. This allows us to recommend stored pipelines without ever executing them.

To achieve that result, we built two datasets, one for classification and for regression respectively, composed of a concatenation between a new learning task and a reference learning task in the database, with as target the performance of the new learning task when executing the reference pipeline.

To train our metalearner, we picked separate datasets from Kaggle [URL9] and OpenML [URL12] from those in the database, 261 for classification and 175 for regression. We ran each dataset on all the classification and regression pipelines in the database.
We saved several performance metrics at first, accuracy and balanced accuracy for classification and mean absolute error, mean squared error and r-squared for regression, we ended up using balanced accuracy for classification and R-squared for regression in our metalearners as they provided the best performance and generalization.

For the implementation of the metalearners, we use a stacked ensemble of XGBoost regression models. These models can predict, based on the concatenation of a new learning task metafeatures and those of a reference learning task in our database, can predict an estimate of the performance (balanced accuracy for classification and r-squared for regression) of the reference learning task top performing pipeline on the new dataset. This allows us to build a ranking of our pipelines stored in the database, the top-performing pipeline is returned to the user in a Javascript Object Notation (JSON) format.

4.1.1.2 Hyperparameter optimization

In this section, we will detail the hyperparameter optimizer of the AutoML solution. This tool can further tune the pipeline chosen by the end-to-end recommendation system to further suit the new dataset's needs, or can perform an automated hyperparameter tuning job for a pipeline designed by the user.

Search space generator Hyperparameter tuning libraries require the user to specify which parameters to tune, as well as their ranges. To ensure the least amount of assistance, we built a tool which maps the pipeline to tune to a set of hyperparameters of multiple types (categorical parameters, parameters with uniform distributions...), with their ranges and possible values. The user doesn't need to specify anything to the tuning tool, only the maximum execution time. This automated extraction of hyperparameters is illustrated in the figure 4.2 below.

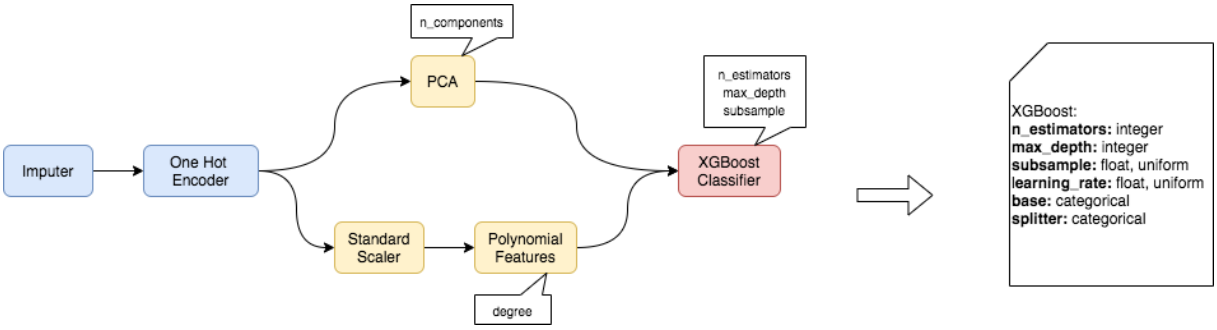


Figure 4.2: Automated generation of the search space from a given pipeline.

Hyperparameter tuner This component runs an informed hyperparameter search across the search space built in the earlier step on our pipeline using Bayesian optimization. In its first iteration, the system behaves like a random hyperparameter search, sampling from the search

space and evaluating the pipeline with the new parameters. These evaluations are however stored to build an acquisition function, that will estimate the unknown objective function. The system also chooses the parameters that will maximise the acquisition function. Also, the more hyperparameter executions which exist in a close region of the search space, the lower the uncertainty of the acquisition function will be.

4.2 Detailed design

After presenting a high-level overview of the system, we will dig deeper in this section into the different components of the system, the different interactions between them in order to fulfill the main requirements.

4.2.1 Sequence diagrams

In this section, we will detail how the system's components responds to the main user scenarios. We will be using sequence diagrams, which provide a clear graphical representation of the different interactions between the user and system's components through the execution time of the use case. We will detail in the following the sequence diagrams of the two main scenarios.

4.2.1.1 End to end pipeline recommendation sequence diagram

The main scenario of our application is illustrated n the figure 4.3, which is the end-to-end recommendation of a machine learning workflow, with its hyperparameters. The user is requested to enter the path or URL of his dataset, as well as specify the target feature and whether the supervised machine learning task is a classification or regression task.
Then, the recommendation process starts. First, we transform the dataset into a more computable format, by extracting its metafeatures. This is the most resource-heavy part of the process, as many calculations are made to extract relevant information for pipeline recommendation, but all operations have a time complexity equivalent to $O(nm)$. Then, we extract from the metalearning database all the metafeatures of datasets corresponding to each reference pipeline. We concatenate the each reference metafeatures to our current dataset metafeatures to build a vector that represents the relationship between our dataset and one that exists in our

database. Each of those vectors is passed to the pipeline selector, which provides an estimate of the performance of a reference pipeline on our new dataset. The top-performing pipeline is selected.

If the predicted performance is lower than a threshold, we run a hyperparameter tuning job on the pipeline before returning it. Then, the pipeline is returned to the user, rendered graphically and trained on the dataset. Optionally, the user can request a hyperparameter tuning job on it.

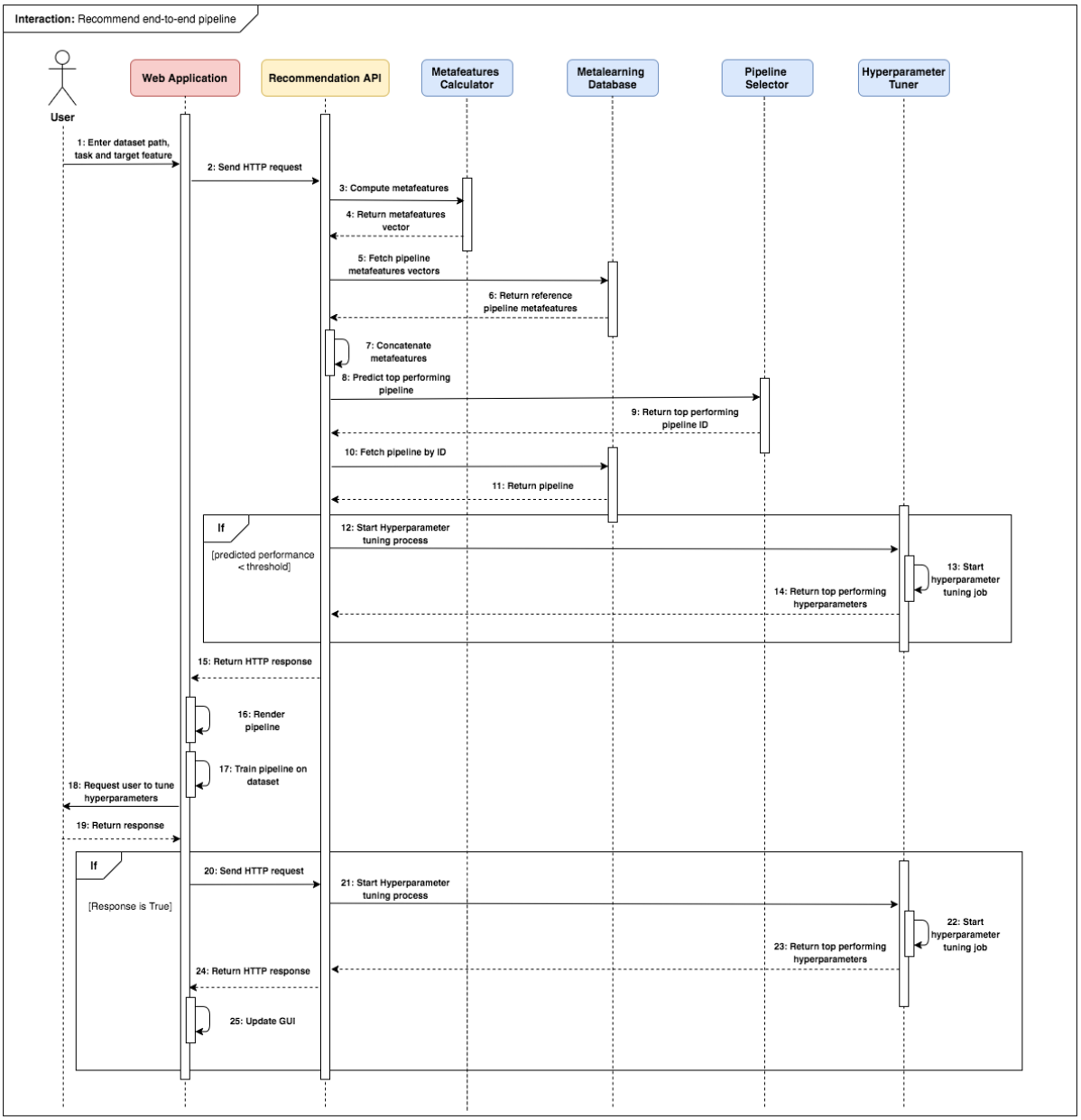


Figure 4.3: End to end pipeline recommendation sequence diagram.

4.2.1.2 Hyperparameter tuning sequence diagram

In the figure 4.4, we present our second use case, tuning a pipeline hyperparameters. The request body is similar to the end-to-end recommendation body, but the user has to also specify a time limit of the hyperparameter tuning job. No other configuration is required.
First, the each block of the pipeline is mapped to a fixed list of hyperparameters to tune, along with their ranges and types. The hyperparameter tuning job starts. At first, the choice of parameters is random, as we cannot build an estimate of the objective function. The pipeline is executed with the new parameters and we keep track of the executions with their performances. As the number of iterations progresses, we can build a better estimate of the objective function, and we use our estimate function, called acquisition function, to choose the parameters that will maximise our performance/minimize our loss, as this technique is an informed search.

Once the time limit is reached, the hyperparameter tuning job is stopped and the best parameters will be returned to the web application, which will display the appropriate changes.

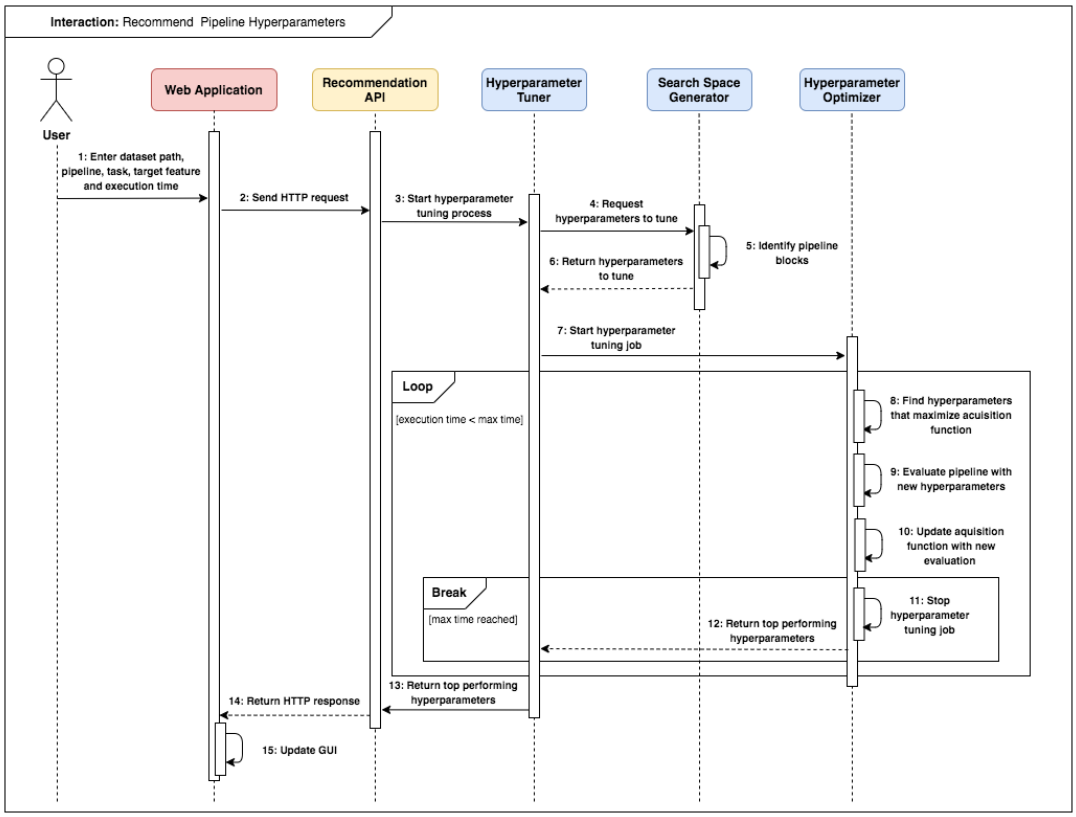


Figure 4.4: Hyperparameter recommendation sequence diagram.

4.2.2 Activity diagram

The UML activity diagram allows us to graphically model the behavior of the system while running the system. In the figure 4.5, we model the activity diagram of the global use case.

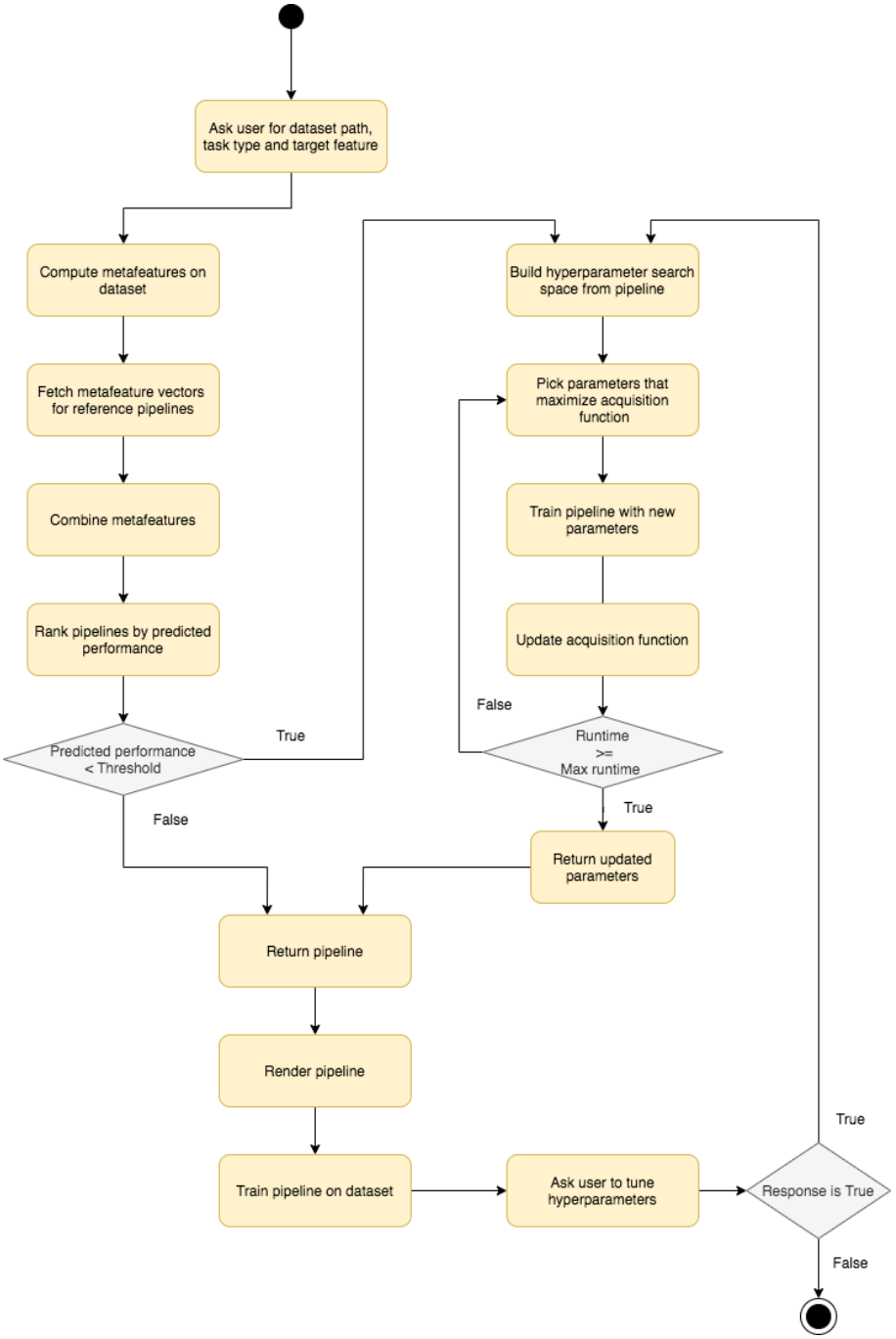


Figure 4.5: Activity diagram.

Conclusion

We have devoted this chapter to a detailed description of our AutoML solution and its various architectural components. The realization of our application will be based on these design choice.

The next chapter will therefore be dedicated to the application side of our work through the presentation of the environment and the technological choices as well as the main interfaces of our application and finally benchmark with reference solutions on the market.

Chapter 5

Achievements

Introduction

This chapter will be dedicated to the development phase of the project. We will describe the work environment, both hardware and software. Then, we will present an overview of the achieved work and compare our solution's performance to rival AutoML frameworks.

5.1 Work Environment

5.1.1 Hardware Environment

During the implementation phase of the project, a personal computer and a cloud virtual machine were used. The table 5.1 presents the hardware characteristics of both machines.

	Personal Laptop	**Google Colab VM**
CPU	Intel(R) Core i5 @ 1.6GHz	2x Intel(R) Xeon(R) CPU @ 2.30GHz
RAM	8GB	12.7 GB
Operating System	MacOS 10.14.6	Linux 4.19.104+
Usage	Documentation redaction Preliminary study Software implementation	Running machine learning experiments to train meta-learner

Table 5.1: Hardware environment.

5.1.2 Software environment

This section describes the software tools used during the design and development phases of this project. They are all presented in the table 5.2. More details are in Appendix A.

Software	**Category**	**Usage**
Microsoft Visual Studio Code	Code editor	Development
Jupyter Notebook	Data science environment	Development & Testing
Postman	API development environment	API testing
Overleaf	Online LaTeX editor	Report redaction
Draw.io	Online diagram tool	Drawing diagrams
Microsoft Teams	Collaboration platform	Remote communication

Table 5.2: Software environment.

5.2 Technological choices

In this section, we present the several technologies used during the development phase of the project, summarized in the table 5.3. More details are in Appendix A

Technology	**Usage**
Python	Main programming language
Flask	Framework for creating web APIs
Scikit-learn	Machine learning library used for building and evaluating ML pipelines
XGBoost	Machine learning libraries for supporting gradient boosting models
LightGBM	
Pandas	Software libraries for data manipulation and scientific computing, used to extract metrics on datasets.
Scipy	
H2O	Machine learning library used for implementing metalearning model
Ray Tune	Python library for large-scale hyperparameter tuning

Table 5.3: Technological choices.

5.3 Results

In this section, we will present an overview of the achieved work. First, we will start by providing a detailed description of the metalearning database and its contents. Then, we will showcase the recommendation process with screenshots of the main application.

5.3.1 Metalearning database

The metalearning database is the starting point of the AutoML solution. It contains 216 classification pipelines and 70 regression pipelines. These pipelines have been curated as the top performing pipelines of selected learning tasks. These pipelines use a variety of learning algorithms from the Scikit-learn, XGBoost and LightGBM frameworks. They are resumed in the tables 5.4 and 5.5 below, for classification and regression.

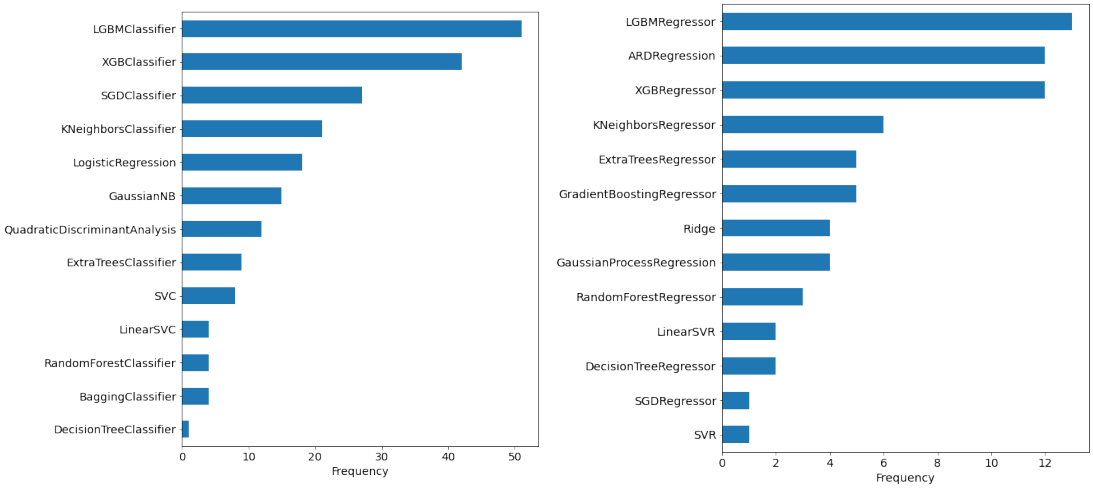
Algorithm Name	**Source**	**# Hyperparameters**
SVC	sklearn	4
LinearSVC	sklearn	2
LogisticRegression	sklearn	-
SGDClassifier	sklearn	7
RandomForestClassifier	sklearn	6
GaussianNB	sklearn	1
KNeighborsClassifier	sklearn	3
BaggingClassifier	sklearn	3
ExtraTreesClassifier	sklearn	6
XGBClassifier	xgboost	9
QuadraticDiscriminantAnalysis	sklearn	2
DecisionTreeClassifier	sklearn	4
LGBMClassifier	lightgbm	6

Table 5.4: Classification algorithms: name, source and number of hyperparameters.

Algorithm Name	**Source**	**# Hyperparameters**
SVR	sklearn	5
LinearSVR	sklearn	3
Ridge	sklearn	2
SGDRegressor	sklearn	6
RandomForestRegressor	sklearn	7
GaussianProcessRegressor	sklearn	1
KNeighborsRegressor	sklearn	3
ExtraTreesRegressor	sklearn	7
GradientBoostingRegressor	sklearn	7
XGBRegressor	xgboost	11
ARDRegression	sklearn	6
DecisionTreeRegressor	sklearn	5
LGBMRegressor	lightgbm	6

Table 5.5: Regression algorithms: name, source and number of hyperparameters.

The figure 5.1 shows that gradient boosting learning algorithms, like XGBoost and LightGBM, are the most frequently used in the database.



(a) Classification algorithms frequencies.

(b) Regression algorithms frequencies.

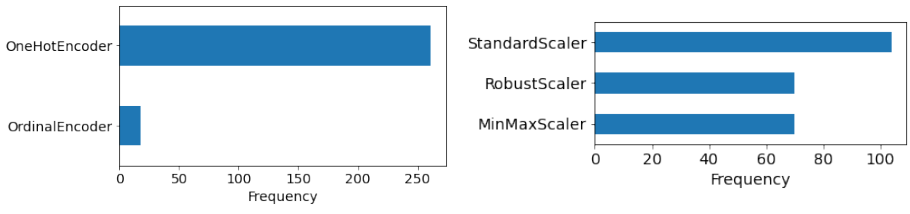
Figure 5.1: Frequencies of classification & regression algorithms in metalearning database.

The pipelines also contain several data processing algorithms, which are described in the table 5.6.

Algorithm Name	**Role**	**Source**	**# Hyperparameters**
SimpleImputer	Missing value imputation	sklearn	1
MinMaxScaler	Feature scalers	sklearn	-
StandardScaler		sklearn	-
RobustScaler		sklearn	-
OrdinalEncoding	Categorical encoding	sklearn	-
OneHotEncoder		sklearn	1
PCA	Dimentionality Reduction	sklearn	2
TruncatedSVD		sklearn	1
PolynomialFeatures	Feature generation	sklearn	3
VarianceThreshold	Feature selection	sklearn	1
FeatureAgglomeration	Clustering	sklearn	3
RBFSampler	Kernel approximation	sklearn	2
Normalizer	Instances normalization	sklearn	-

Table 5.6: Data processing algorithms: name, role, source and number of hyperparameters.

Statistics ran on the stored pipelines also show, as illustrated in the figure 5.2, that OneHotEncoding is the most used encoding algorithms, while scaling algorithms like StandardScaling, MinMaxScaling and RobustScaling are used similarly across all pipelines.



(a) Encoding algorithms frequencies.

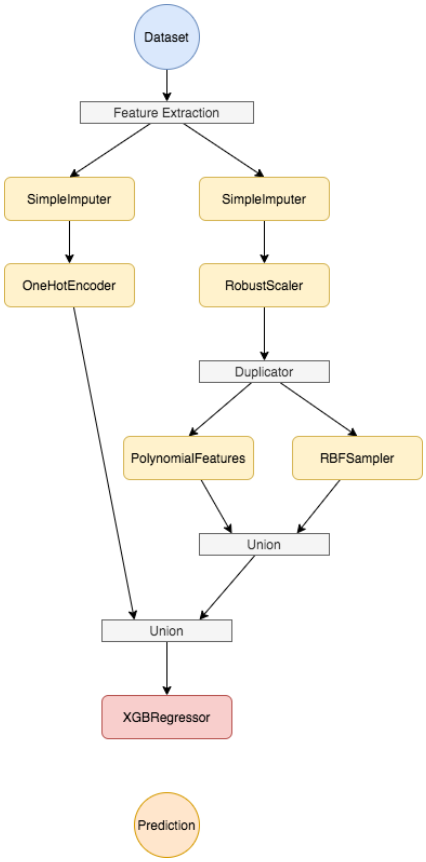
(b) Scaling algorithms frequencies.

Figure 5.2: Frequencies of usage of encoding & scaling algorithms in metalearning database.

The pipelines stored in the database are serialized in a JSON format, which ensures interoperability between frameworks. The figure 5.3 compares a pipeline stored in the database to its graphical representation.

```
{
  "name":"",
  "steps":[
    {
      "name":"columntransformer",
      "transformers":[
        {
          "transformer":{
            "name":"pipeline-1",
            "steps":[
              {
                "name":"simpleimputer",
                "params":{ }
              },
              {
                "name":"robustscaler",
                "params":{ }
              },
              {
                "name":"featureunion",
                "transformers":[
                  {
                    "name":"polynomialfeatures",
                    "params":{ }
                  },
                  {
                    "name":"rbfsampler",
                    "params":{ }
                  }
                ]
              }
            ]
          },
          "column_selector":"number"
        },
        {
          "transformer":{
            "name":"pipeline-2",
            "steps":[
              {
                "name":"simpleimputer",
                "params":{ }
              },
              {
                "name":"onehotencoder",
                "params":{ }
              }
            ]
          },
          "column_selector":"category"
        }
      ],
      "remainder":"drop"
    },
    {
      "name":"xgbregressor",
      "params":{ }
    }
  ]
}
```


(a) Regression pipeline stored in database.



(b) Graphical representation of the same pipeline.

Figure 5.3: Format for storing pipelines in the database.

This generic format is necessary since we were running our AutoML experiments in scikit-learn due to its ease of use and rich library, while the production application which uses the recommender system, HazeML, uses SparkML for better parallelization for big data tasks.

5.3.2 Illustrations from the Application

The AutoML recommendation system resides within a code-less machine learning editor, named HazeML, designed to simplify building and running machine learning workflows.

The figure 5.4 demonstrates the main user interface of the project, where a user can create a machine learning workflow from scratch, edit an existing workflow or run a recommendation request.

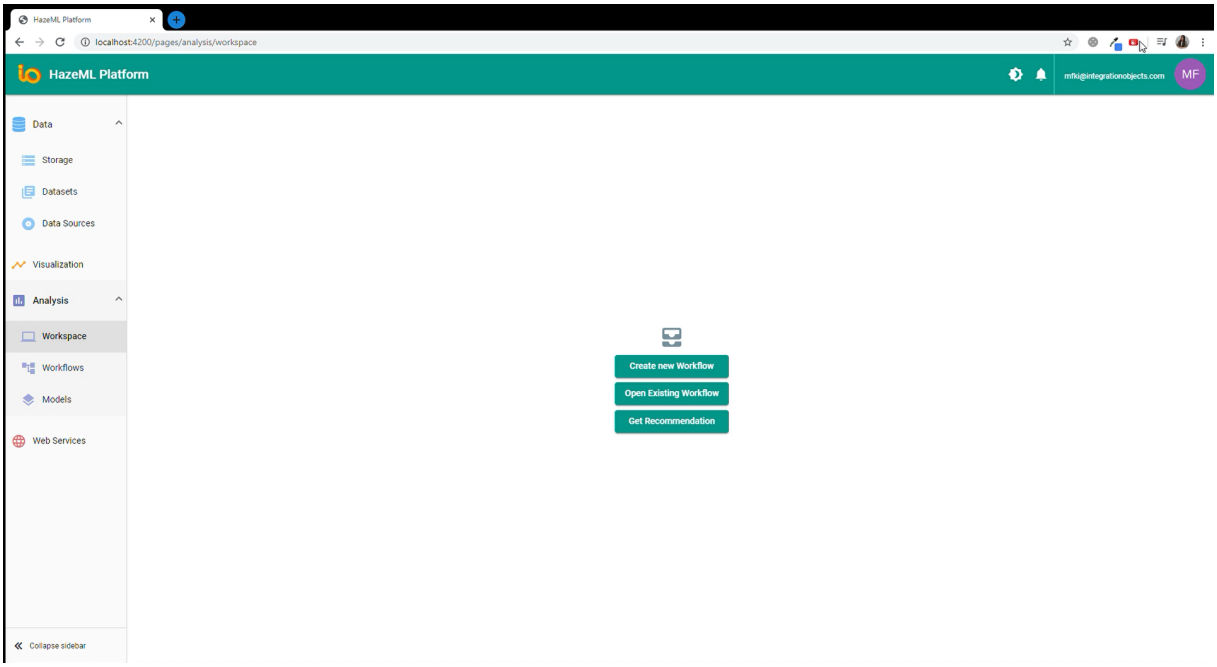


Figure 5.4: Machine learning editor web application.

Then, the user is requested to enter relevant information for the recommender system, by filling a form presented in the figure 5.5 below.

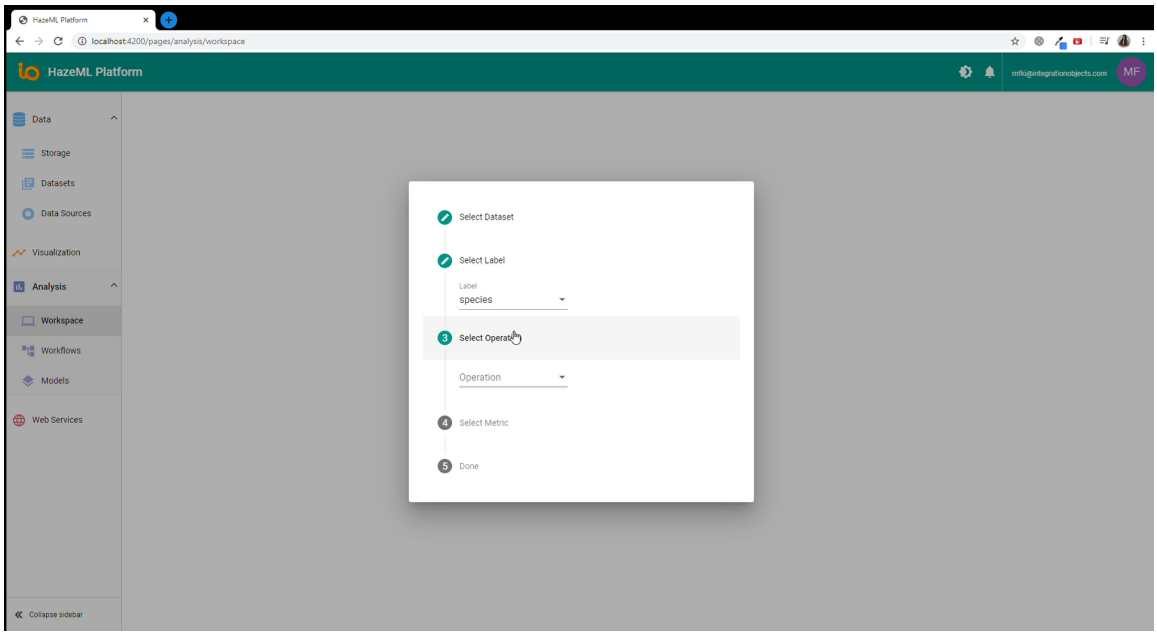


Figure 5.5: Entering information about learning task for recommendation.

The recommended pipeline is illustrated in the figure 5.6 below. It is composed of a series of data processing algorithms (Imputer and One Hot Encoder applied on categorical features and Imputer and PCA applied to numerical features in our case) and a learning algorithm (K-Nearest Neighbors in this example).

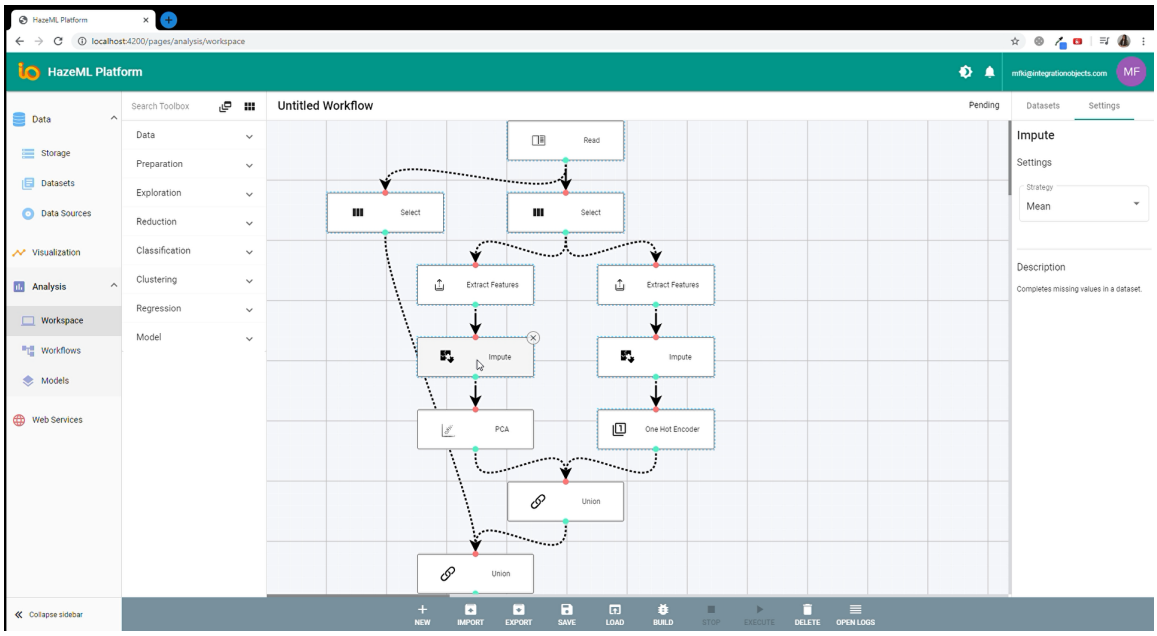


Figure 5.6: Graphical representation of the machine learning pipeline.

The right-side panel in the figure 5.7 contains all the necessary blocks for building a machine learning pipeline, categorized based of their type. The left-side panel is dedicated to selecting hyperparameters of a certain machine learning block. Commands reside in the bottom of the user interface for running the workflow or saving it for further editing.

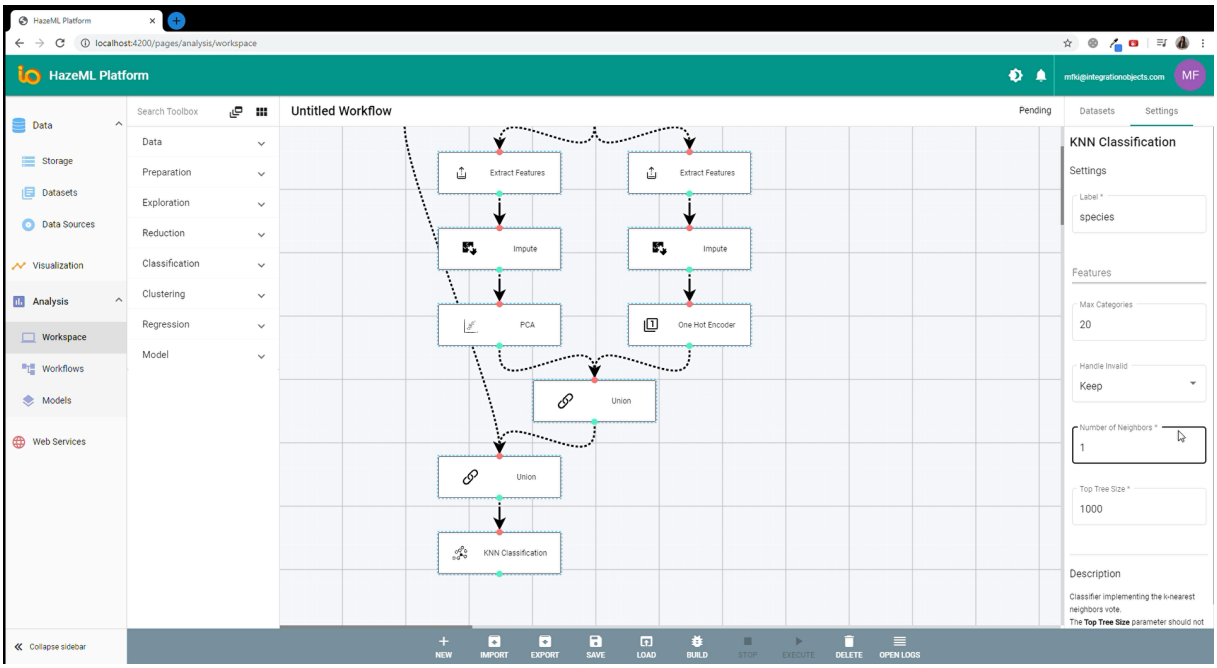


Figure 5.7: Editing hyperparameters of a pipeline block graphically.

5.4 Benchmarks

To test our the performance of our AutoML framework, we picked two rival solutions, namely H2O AutoML and Microsoft ML.NET AutoML CLI. We chose different datasets from Kaggle [URL9] for comparison and reported the results in classification and regression in the following bemchmarks.

During this benchmark, we will use Microsoft [URL1] and H2O [URL2] AutoML solutions at default settings (1 hour maximum runtime, 5-fold cross validation). We will compare them using several performance metrics and duration. For our solution, the duration is the sum between the response time of our AutoML recommender system with its execution time while performing 5-fold cross-validation (Since the recommender system returns a pipeline without executing it, contrary to the other solutions which return a trained model or pipeline).

5.4.1 Classification benchmarks

5.4.1.1 Metrics

In this classification benchmark, we will use the following performance metrics to compare the AutoML frameworks. We will note that Microsoft ML.NET AutoML CLI, being closed source, doesn't return to us Log Loss for comparison, and doesn't display balanced accuracies during binary classification tests.

- **Accuracy:** the proportion of true results among the total number of cases examined. Higher is better.
- **Balanced Accuracy:** used to deal with imbalanced datasets. It is defined as the average of recall obtained on each class. Higher is better.
- **Area Under ROC Curve (AUC):** performance metric that tells how much a classification model is capable of distinguishing between classes. Higher is better.
- **F1-Score:** weighted average of the precision and recall. Higher is better.
- **Log Loss:** performance metric that measures the uncertainty of the probabilities of a classification model by comparing them to the true labels. Lower is better.

5.4.1.2 Datasets

The table 5.7 summarizes the Kaggle datasets used during this classification benchmark. It contains information about their shape and class distribution.

Dataset name	# Samples	# Features	# Num.	# Cat.	# Classes	Maj. Class	Min. Class
Heart Disease UCI	303	13	6	7	2	0.5445	0.4555
Airline Passenger Satisfaction	103904	22	18	4	2	0.56	0.44
Pulsar Star	17952	8	8	0	2	0.9057	0.0943
Online Shopper's Intention	12330	17	10	7	2	0.8453	0.1547
Forest Cover Type	15120	54	10	44	7	0.1428	0.1428
Profitable Customer Segments	6620	70	65	5	3	0.4646	0.2518

Table 5.7: Classification datasets: Name, Number of samples, of features (categorical and numerical), of classes and majority and minority class ratios.

5.4.1.3 Results

Binary classification benchmarks This section summarizes the benchmark across five classification Kaggle datasets.

In the first example in the table 5.8, we notice that our solution already outperforms Microsoft ML.NET AutoML, while offering a result 1058 times faster. After 15 minutes of hyperparameter tuning, our solution performs similarly to the best performing framework, H2O AutoML.

AutoML Tool	Accuracy	Balanced Acc	AUC	F1	Log Loss	Time (s)
H2O AutoML	**0.86798**	**0.86377**	0.9122	**0.8833**	**0.3729**	3539
IO AutoML (15m HPO)	0.86478	0.85878	**0.91711**	0.88116	0.37623	904
IO AutoML (without HPO)	0.84482	0.84421	0.90681	0.85763	0.41516	**3.348**
Microsoft ML.NET AutoML	0.8438	-	0.8438	0.9	-	3600

Table 5.8: Benchmark on Heart Disease UCI dataset.

In this second test on the Airline Passenger Satisfaction dataset, summarized in the ta-

ble 5.9, while rival solutions provided better performance, our solution was able to provide a pipeline at 127 times faster. Following a 30 minute hyperparameter run, it obtained a more comparable score.

AutoML Tool	Accuracy	Balanced Acc	AUC	F1	Log Loss	Time (s)
Microsoft ML.NET AutoML	**0.9685**	**-**	**0.9961**	**0.9628**	**-**	3600
H2O AutoML	0.96542	**0.9631**	0.99551	0.95952	0.08197	3444
IO AutoML (30m HPO)	0.9528	0.95037	0.98962	0.93803	0.1088	1828
IO (without HPO)	0.94199	0.9388	0.98743	0.93182	0.14939	**27.1**

Table 5.9: Benchmark on Airline Passenger Satisfaction dataset.

In the following test on the Pulsar Lights dataset, as seen in the table 5.10, our solution provided better performance than H2O AutoML without hyperparameter optimization, and at 200 times faster rate. Unfortunately, this was one of the instances where hyperparameter optimization failed to improve performance in its 30 minute run.

AutoML Tool	Accuracy	Balanced Acc	AUC	F1	Log Loss	Time (s)
Microsoft ML.NET AutoML	**0.9829**	**-**	**0.9794**	**0.9096**	**-**	3600
IO (Without HPO)	0.98028	**0.92113**	0.97755	0.88739	0.08955	**14.4**
H2O AutoML	0.97882	0.85637	0.97762	0.88114	0.07517	2883
IO (30m HPO)	No Improvement					1816

Table 5.10: Benchmark on Pulsar Stars dataset.

In this last binary classification test on the Online Shopper's Intention dataset, summarized in the table 5.11, our solution failed to perform better than H2O and Microsoft, even if it provided a strong pipeline 113 times faster. Like the previous test, a 30 minute hyperparameter tuning run failed to improve performance.

AutoML Tool	Accuracy	Balanced Acc	AUC	F1	Log Loss	Time (s)
Microsoft ML.NET AutoML	**0.9685**	**-**	**0.9409**	**0.6885**	**-**	3600
H2O AutoML	0.89903	**0.82922**	**0.93372**	0.69084	0.22456	2967
IO (Without HPO)	0.89603	0.75878	0.90185	0.62464	0.33886	**26.1**
IO (30m HPO)	No Improvement					1827

Table 5.11: Benchmark on Online Shopper's Intention dataset.

Multi-class classification Below are benchmark results across two multi-class classification Kaggle datasets. Since Area Under Curve (AUC) and F-1 Score are binary-only classification metrics, they will be ignored in the following benchmarks.

In this first multi-class classification benchmark on the Forest Cover Type dataset, our initial recommended solution performed worse than Microsoft and H2O AutoML frameworks, but the solution was returned and trained at a 27 times smaller delay. A 30 minute hyperparameter tuning run helped reduce the gap between the frameworks. The results are summarized in the table 5.12.

AutoML Tool	Accuracy	Balanced Acc	Log Loss	Time (s)
Microsoft ML.NET AutoML	**0.8973**	**0.895**	**-**	3600
H2O AutoML	0.88313	0.88323	**0.418**	3532
IO (30m HPO)	0.83955	0.839684	0.424	1936
IO (Without HPO)	0.8119	0.81211	0.6327	**130**

Table 5.12: Benchmark on Forest Cover Type dataset.

In this last classification test, summarized in the table 5.13, all three AutoML frameworks provided a mediocre performance, with our solution offering the lower performance score, but was 23 times faster than competitors. Once again, the hyperparameter tuner failed to improve the score in a 30 minute timeframe.

AutoML Tool	Accuracy	Balanced Acc	Log Loss	Time (s)
H2O AutoML	**0.59905**	**0.53161**	**0.91663**	3203
Microsoft ML.NET AutoML	0.5954	0.5305	**-**	3600
IO (Without HPO)	0.56375	0.49847	0.97921	**135**
IO (30m HPO)	No Improvement			1936

Table 5.13: Benchmark on Profitable Customer Segments dataset.

5.4.2 Regression benchmarks

5.4.2.1 Metrics

For the regression benchmark, we will use the following performance metrics for comparison:

- **Mean Absolute Error (MAE):** the absolute difference between the target value and the value predicted by the regression model. Lower is better.
- **Mean Squared Error (MSE):** the average of the squared difference between the target value and the value predicted by the model. Lower is better.
- **Root Mean Squared Error (RMSE):** the square root of the averaged squared difference between the target value and the value predicted by the model. Lower is better.
- **R-Squared:** a metric that compares the current model with a constant baseline and tells us how much our model is better. The constant baseline is chosen by taking the mean of the data. R-Squared is a scale-free score that implies it doesn't matter whether the values are too large or too small, the R-Squared will always be less than or equal to 1. Higher is better.

5.4.2.2 Datasets

The table 5.14 summarizes the Kaggle datasets used during regression benchmarks. It contains information about their shape and feature ratios.

Dataset name	# Samples	# Features	# Num. Features	# Cat. Features
Greener Manufacturing	4209	375	0	375
Bike Sharing Demand	10886	10	6	4
House Pricing	1460	79	33	46
Russian Housing Market	30471	290	275	15

Table 5.14: Regression datasets: Name, Number of samples and features (categorical and numerical).

5.4.2.3 Results

In this section we compare how our solution performs compared to two rival AutoML frameworks across 3 Kaggle regression datasets. We will note that tests with hyperparameter tuning were not performed in regression tests.

In this first test on the Green Manufacturing dataset, we can notice that our system performs in between Microsoft and H2O AutoML frameworks, while providing a solution 74 times faster than the two rivals. The scores are in the table 5.15 below.

AutoML Tool	MAE	MSE	RMSE	R-Squared	Time (s)
Microsoft ML.NET AutoML	**5.04**	**64.04**	**8**	**0.5774**	3600
IO AutoML (Without HPO)	5.33947	70.94644	8.35927	0.56415	**46.1**
H2O AutoML	5.37872	71.41517	8.45075	0.55568	3235

Table 5.15: Benchmark on Green Manufacturing dataset.

In this second test on the Bike Sharing dataset, summarized in the table 5.16, we can notice that our solution similarly in metrics like R-Squared in 331 times faster response, and worse in other metrics compared to Microsoft and AutoML frameworks. We will need to perform more tests to see if hyperparameter tuning can boost results.

AutoML Tool	MAE	MSE	RMSE	R-Squared	Time (s)
H2O AutoML	**0.98149**	**3.90538**	**1.9762**	**0.99988**	3446
Microsoft ML.NET AutoML	1.55	8.89	2.98	0.9997	3600
IO AutoML (Without HPO)	3.24863	27.08099	5.20215	0.99917	**10.4**

Table 5.16: Benchmark on Bike Sharing dataset.

In the next test on the House Pricing Kaggle dataset, summarized in the table 5.17, our solution was as intended faster than rivals by about 244 times, but scored lower performance metrics.

AutoML Tool	MAE	MSE	RMSE	R-Squared	Time (s)
Microsoft ML.NET AutoML	16427	587841293.32	24245.44	**0.9119**	3600
H2O AutoML	**15342.415**	**705033274.313**	**26552.463**	0.88821	3446
IO AutoML (Without HPO)	17247.864	834935723.460	196641.414	0.86932	**14.1**

Table 5.17: Benchmark on House Pricing dataset.

In this final regression test, our solution performed better than H2O and worse than Microsoft AutoML frameworks, while being 6.6 times faster, as seen in the table 5.18. We should note that most of this runtime was dedicated to training the pipeline using 5-fold cross-validation, and that the pipeline was returned in 13.8 seconds.

AutoML Tool	MAE	MSE	RMSE	R-Squared	Time (s)
H2O AutoML	**1359790.95**	**7065581169223.14**	**2658116.09**	**0.69077**	3497
IO AutoML (Without HPO)	1402474.43	7432651037300.67	2723513.24	0.67448	**536.8**
Microsoft ML.NET AutoML	1552960.65	9835539164212.21	3136166.32	0.6309	3600

Table 5.18: Benchmark on Russian Housing Market dataset.

5.5 Project planning

The project timeline is presented in the figure 5.8. This Gantt diagram describes the different development phases undertaken during the five month internship at Integration Objects.

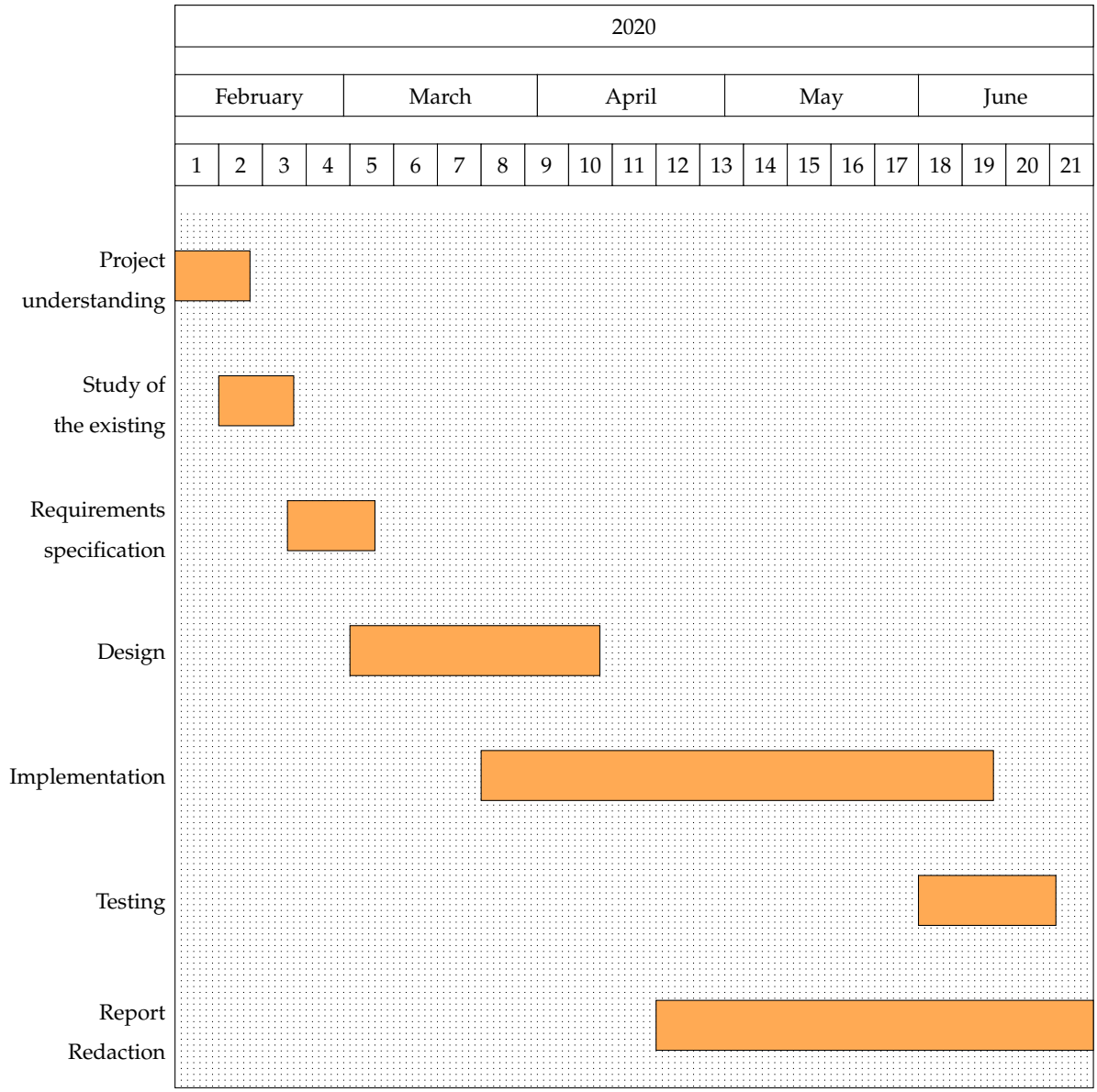


Figure 5.8: Project planning.

Conclusion

During this chapter, we presented our work environment and technological choices, provided an overview of our achievements and compared our solution to leading AutoML frameworks. This chapter allowed us to describe the concrete realizations made during this project.

General Conclusion

As we are living in an artificial intelligence-driven world, automation has become a crucial consideration when working on a data science project. In order to keep up with the increasing demand of such solutions, we need simple ways to deliver solutions to machine learning tasks.

The aim of this graduation internship was to explore the state of the art in AutoML, from its techniques to its shortcomings, and to design a solution tailored for projects at Integration Objects and accessible to its clients. We tried to provide a solution with a strong trade-off between state of the art performance, and fast response time.

Throughout this project report, we have detailed the different steps of our project's lifecycle, starting from a preliminary study where we explored the different techniques used to solve and optimize learning tasks, as well as compared existing AutoML frameworks to further understand our solution's positioning. Then, there was the requirements analysis & specification phase, where we presented our application needs through functional and non-functional requirements. Then, we presented the design of our project where we identified the main components of our application and specified their interactions through sequence diagrams. Finally, we have established the implementation phase where we presented the achieved work and compared our solution to industry leaders.

We faced various challenges when developing this project. First, we found difficulties in finding a metalearning database, and even began building our own before settling on the current setup. Even after finding a metalearning database, we spend a huge part of the internship adapting it to our context. Running evaluations for training the metalearner was also a

challenge that was time consuming. Finally, settling on a metalearner that would generalize well to multiple learning tasks was a challenge, and required trying multiple combinations of metafeatures and figuring out the right architecture of the metalearning model.

It goes without saying that there is always room for improvement. For future iterations, we hope to further improve the performance-efficiency trade-off. We will try to find ways to optimize the metafeature extraction phase to reduce delay, and try to increase the size of the metalearning database as well as the underlying machine learning experiments to improve model diversity and thus performance. Also, we will further improve the performance of the hyperparameter tuning system, or scrap it for a block recommendation engine.

Bibliography

[1] W. Banzhaf. Artificial intelligence: Genetic programming. 2015.

[2] I. Drori, Y. Krishnamurthy, R. Rampin, R. Lourenço, J. One, K. Cho, C. Silva, and J. Freire. Alphad3m: Machine learning pipeline synthesis. In AutoML Workshop at ICML, 2018.

[3] M. Feurer, A. Klein, K. Eggensperger, J. T. Springenberg, M. Blum, and F. Hutter. Auto-sklearn: efficient and robust automated machine learning. In Automated Machine Learning, pages 113–134. Springer, Cham, 2019.

[4] Y. Heffetz, R. Vainstein, G. Katz, and L. Rokach. Deepline: Automl tool for pipelines generation using deep reinforcement learning and hierarchical actions filtering. arXiv preprint arXiv:1911.00061, 2019.

[5] F. Hutter, L. Kotthoff, and J. Vanschoren, editors. Automatic Machine Learning: Methods, Systems, Challenges. Springer, 2019.

[6] R. S. Olson and J. H. Moore. Tpot: A tree-based pipeline optimization tool for automating machine learning. In Workshop on automatic machine learning, pages 66–74, 2016.

[7] Z. Shang, E. Zgraggen, P. Eichmann, and T. Kraska. Niseko: a large-scale meta-learning dataset. epsilon, 1:4, 2019.

[8] R. S. Sutton and A. G. Barto. Reinforcement Learning: An Introduction. A Bradford Book, Cambridge, MA, USA, 2018.

[9] C. Thornton, F. Hutter, H. H. Hoos, and K. Leyton-Brown. Auto-weka: Combined selection and hyperparameter optimization of classification algorithms. In Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining, pages 847–855, 2013.

[10] J. Vanschoren. Meta-learning. In Automated Machine Learning, pages 113–134. Springer, Cham, 2019.

[11] Q. Yao, M. Wang, H. J. Escalante, G. Isabelle, Y.-Q. Hu, L. Yu-Feng, W.-W. Tu, Y. Qiang, and Y. Yang. Taking human out of learning applications: A survey on automated machine learning. 2018.

Netography

[URL1] Automate model training with the ml.net cli. https://docs.microsoft.com/en-us/dotnet/machine-learning/automate-training-with-cli. [Online; accessed 30-May-2020].

[URL2] Automl: Automatic machine learning - h2o 3.30.0.6 documentation. http://docs.h2o.ai/h2o/latest-stable/h2o-docs/automl.html. [Online; accessed 02-April-2020].

[URL3] AutoML workshop @ ICML'14 . https://sites.google.com/site/automlwsicml14/. [Online; accessed 12-April-2020].

[URL4] Best Jobs in America 2019 . https://www.glassdoor.com/List/Best-Jobs-in-America-2019-LST_KQ0,25.htm. [Online; accessed 01-March-2020].

[URL5] Computational complexity of machine learning algorithms. https://www.thekerneltrip.com/machine/learning/computational-complexity-learning-algorithms/. [Online; accessed 04-April-2020].

[URL6] Feature engineering, explained . https://www.kdnuggets.com/2018/12/feature-engineering-explained.html. [Online; accessed 22-March-2020].

[URL7] Genetic algorithms. https://medium.com/datadriveninvestor/genetic-algorithms-9f920939f7cc. [Online; accessed 03-March-2020].

[URL8] How to Choose a Feature Selection Method For Machine Learning . https://machinelearningmastery.com/

feature-selection-with-real-and-categorical-data/. [Online; accessed 20-March-2020].

[URL9] Kaggle: Your home for data science. https://www.kaggle.com/. [Online; accessed 04-June-2020].

[URL10] Lets talk bayesian optimization. https://mlconf.com/blog/lets-talk-bayesian-optimization/. [Online; accessed 20-February-2020].

[URL11] Machine Learning Pipeline Deployment and Architecture . https://www.xenonstack.com/insights/machine-learning-pipeline/. [Online; accessed 13-February-2020].

[URL12] Openml. machine learning, better, together. https://www.openml.org/. [Online; accessed 09-April-2020].

[URL13] Shallow understanding on bayesian optimization. https://towardsdatascience.com/shallow-understanding-on-bayesian-optimization-324 [Online; accessed 15-February-2020].

[URL14] Stacking models for improved predictions. https://www.kdnuggets.com/2017/02/stacking-models-imropved-predictions.html. [Online; accessed 12-April-2020].

[URL15] Steps for data cleaning and why it matters . https://www.geotab.com/blog/data-cleaning/. [Online; accessed 24-March-2020].

Appendix A

Software & Technologies

In this chapter we will further detail the different software environments and technologies used during the project.

A.1 Software environment

- **Microsoft Visual Studio Code:** a free and open source-code editor, developed by Microsoft and released in 2015 for different operating systems. It supports code refactoring, syntax highlighting, snippets and debugging.
- **Jupyter Notebook:** an open-source data science environments that allows users to create and share documents that contain live code, equations, visualizations and narrative text.
- **Postman:** an API Development Environment that helps users to build, test, document, monitor and publish documentation for their APIs.
- **Overleaf:** a collaborative cloud-based LaTeX editor used for writing, editing and publishing scientific documents. It partners with a wide range of scientific publishers to provide official journal LaTeX templates, and direct submission links.
- **Draw.io:** a free online service for scientific and business diagrams and chart drawing. It provides tool-kits for unified modeling language diagrams, flow chart diagrams and many other custom diagrams.

- **Microsoft Teams:** a unified communication and collaboration platform that combines persistent workplace chat, video meetings, screen sharing, file storage (including collaboration on files), and application integration. The service integrates with the Office 365 subscription office productivity suite and features extensions that can integrate with non-Microsoft products.

A.2 Technologies

- **Python:** a dynamically typed and multi-paradigm programming language. It provides supports object-oriented, functional, reflective and imperative prggramming. Designed by Guido van Rossum and developed by the non-profit Python Software Foundation with high-level tools and simple syntaxes, Python enables clear programming on both small and large scales.

- **Flask:** a micro web framework written in Python. It is classified as a microframework because it does not require particular tools or libraries.

- **Scikit-learn:** a machine learning library for Python that provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use. The library is built upon the SciPy scientific programming library.

- **XGBoost:** an open-source software library which provides a gradient boosting framework for C++, Java, Python, R, Julia, Perl, and Scala. It works on Linux, Windows, and macOS. From the project description, it aims to provide a "Scalable, Portable and Distributed Gradient Boosting (GBM, GBRT, GBDT) Library".

- **LightGBM:** a gradient boosting framework that uses tree based learning algorithms. It is designed to be distributed and efficient with the following advantages: faster training speed and higher efficiency, lower memory usage, better accuracy, support of parallel and GPU learning and ability to handlelarge-scale data.

- **Pandas:** a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language. It offers data structures and operations for manipulating numerical tables and time series.
- **Scipy:** a free and open-source Python library used for scientific computing and technical computing. SciPy contains modules for optimization, linear algebra, integration, interpolation, special functions, FFT, signal and image processing, ODE solvers and other tasks common in science and engineering.
- **H2O:** an open source, in-memory, distributed, fast, and scalable machine learning and predictive analytics platform that allows data scientists to build machine learning models on big data and provides easy productionalization of those models in an enterprise environment. H2O's core code is written in Java.
- **Ray Tune:** a Python library that accelerates hyperparameter tuning by allowing users to leverage cutting edge optimization algorithms at scale. Tune provides distributed asynchronous optimization out of the box, and can scale a hyperparameter search from a single machine to a large distributed cluster. It includes state of the art algorithms including (but not limited to) ASHA, BOHB, and Population-Based Training, and integrates with many optimization libraries such as Ax/Botorch, HyperOpt, and Bayesian Optimization. Tune supports any machine learning framework, including PyTorch, TensorFlow, XGBoost, LightGBM, and Keras.

Abstract

The rising demand in machine learning solutions has created the need to automate the work of data scientists. This created a huge surge in a field now called AutoML. In this graduation project, we designed & developed an AutoML recommender system to integrate with a graphical machine learning pipelines designer. This system leverages meta-learning techniques to recommend solutions efficiently, and uses Bayesian optimization for running automated hyperparameter tuning jobs.
Keywords: AutoML, Meta-Learning, Hyperparameter tuning, Bayesian optimization, Machine learning pipeline.

Résumé

La demande croissante de solutions d'apprentissage automatique a créé le besoin d'automatiser le travail des analystes de données. Cela a créé un grand intérêt dans le domaine d'AutoML. Dans ce projet de fin d'études, nous avons conçu et développé un système de recommandation AutoML à intégrer dans une application web de creation de pipelines d'apprentissage automatique graphique. Ce système exploite les techniques de méta-apprentissage pour recommander des solutions de manière efficace et utilise l'optimisation Bayésienne pour exécuter des travaux de réglage hyperparamétrique automatisés.
Mots-clés: AutoML, méta-apprentissage, réglage des hyperparamètres, optimisation Bayésienne, pipeline d'apprentissage automatique.

ملخص

خلق الطلب المتزايد على حلول التعلم الآلي الحاجة لأتمتة عمل مهندسي علوم البيانات. وقد خلق هذا اهتمامًا كبيرًا بمجال AutoML . في مشروع ختم الدراسات ، قمنا بتصميم وتنفيذ نظام مقترح AutoML للتكامل مع تطبيق ويب لسير عمل التعلم الآلي. يستفيد هذا النظام من تقنيات Meta-Learning لإيجاد حلول بسرعة أكبر، ويستخدم التحسين البايزي لضبط المعلمات الآلي.
الكلمات المفاتيح : AutoML ، Metalearning ، تحسين المعامل الفائق ، التحسين البايزي ، خطوط أنابيب التعلم الآلي.